A Project Report on

NirmalSagar: Underwater Marine Litter & Plastic Detection

Submitted in partial fulfillment of the requirements for the award
of the degree of

Bachelor of Engineering

in

Computer Science and Engineering (Data Science)

by

Aditya Kate(23107126)
Suman Manik(23107056)
Tanmay Harmalkar(23107099)
Abdullah Khan(23107119)

Under the Guidance of

Dr. Veena Trivedi
Ms. Swati Parhad



Computer Science and Engineering (Data Science)
A.P. Shah Institute of Technology
G.B.Road,Kasarvadavli, Thane(W)-400615
UNIVERSITY OF MUMBAI

Academic Year 2026-2027

Approval Sheet

This Project Report entitled ***NirmalSagar: Underwater Marine Litter & Plastic Detection*** Submitted by ***Aditya Kate*** (23107126), ***Suman Manik*** (23107056), ***Tanmay Harmalkar*** (23107099), ***Abdullah Khan*** (23107119) is approved for the partial fulfillment of the requirement for the award of the degree of ***Bachelor of Engineering*** in ***Computer Science and Engineering (Data Science)*** from ***University of Mumbai***.

Dr. Veena Trivedi
Guide

Ms. Swati Parhad
Co-Guide

Dr. Pravin Adivarekar
HOD, Computer Science and Engineering (Data Science)

Place: A.P. Shah Institute of Technology, Thane

Date:

CERTIFICATE

This is to certify that the project entitled ***NirmalSagar: Underwater Marine Litter & Plastic Detection*** submitted by ***Aditya Kate*** (23107126), ***Suman Manik*** (23107056), ***Tanmay Harmalkar*** (23107099), ***Abdullah Khan*** (23107119) for the partial fulfillment of the requirement for award of a degree ***Bachelor of Engineering*** in ***Computer Science and Engineering (Data Science)*** to the University of Mumbai, is a bonafide work carried out during academic year 2026-2027.

Dr. Veena Trivedi
Guide

Ms. Swati Parhad
Co-Guide

Dr. Pravin Adivarekar
HOD, C.S.E (Data Science)

Dr. Uttam D. Kolekar
Principal

External Examiner(s)

1.

2.

Internal Examiner(s)

1.

2.

Place: A.P. Shah Institute of Technology, Thane

Date:

Acknowledgement

We have great pleasure in presenting the report on **NirmalSagar: Underwater Marine Litter & Plastic Detection.** We take this opportunity to express our sincere thanks towards our guide **Dr. Veena Trivedi** & Co-Guide **Ms. Swati Parhad** for providing the technical guidelines and suggestions regarding line of work. We would like to express our gratitude towards their constant encouragement, support and guidance through the development of project.

We thank **Dr. Pravin Adivarekar**, Head of Department, for his encouragement during the progress meeting and for providing guidelines to write this report.

We express our gratitude towards BE project co-ordinators, for being encouraging throughout the course and for their guidance.

We also thank the entire staff of APSIT for their invaluable help rendered during the course of this work. We wish to express our deep gratitude towards all our colleagues of APSIT for their encouragement.

Aditya Kate
(23107126)

Suman Manik
(23107056)

Tanmay Harmalkar
(23107099)

Abdullah Khan
(23107119)

Declaration

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, We have adequately cited and referenced the original sources. We also declare that We have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(Signature)
Aditya Kate (23107126)

(Signature)
Suman Manik (23107056)

(Signature)
Tanmay Harmalkar (23107099)

(Signature)
Abdullah Khan (23107119)

Date:

Abstract

Marine debris such as plastic waste, fishing nets, bottles, and other submerged objects is a major concern for marine life and underwater ecosystems. Detection of such debris through manual monitoring is time-consuming and requires significant human effort. Moreover, underwater conditions such as turbidity, low light, poor visibility, lighting variations, and complex backgrounds make automated debris detection challenging. The project proposes an AI-based computer vision system using deep learning for the automated detection and localization of submerged marine debris from underwater images. The proposed detection pipeline combines underwater image enhancement and YOLO-based object detection for identifying marine litter in challenging underwater scenes. The system also focuses on domain adaptation and generalisation strategies, including synthetic augmentation, style transfer, and adversarial domain adaptation, to improve the ability of models trained on datasets such as TrashCan 1.0 and JAMSTEC to generalise to unseen and murkier Indian coastal and seafloor conditions. The project further aims to benchmark the model across multiple heterogeneous underwater marine-debris datasets using standard evaluation metrics such as mAP, IoU, and F1 score. A debris-density estimation module is also proposed to aggregate per-frame detections into an area-level pollution score, supporting marine pollution monitoring and cleanup prioritisation along Indian coastlines. The project uses Python, Deep Learning, Computer Vision, OpenCV, image processing, and YOLOv11 for developing and adapting the detection pipeline.

Contents

1 Introduction **2**
- 1.1 Motivation . . . 3
- 1.2 Problem Statement . . . 4
- 1.3 Objectives . . . 4
- 1.4 Scope . . . 5

2 Literature Review **7**
- 2.1 Comparative Analysis of Recent Studies . . . 7
- 2.2 Research Gap . . . 8

3 Project Design **11**
- 3.1 Existing System . . . 11
- 3.2 Proposed System . . . 12
- 3.3 Critical Components of System Architecture . . . 13
  - 3.3.1 Data Acquisition . . . 13
  - 3.3.2 Data Preparation . . . 13
  - 3.3.3 YOLOv11 Model Training . . . 14
  - 3.3.4 Trained Model . . . 14
  - 3.3.5 Inference and Detection . . . 14
  - 3.3.6 Output and Applications . . . 15
- 3.4 System Diagrams . . . 15
  - 3.4.1 UML Diagram . . . 16
  - 3.4.2 Activity Diagram . . . 16
  - 3.4.3 Use Case Diagram . . . 17
  - 3.4.4 Sequence Diagram . . . 18

4 Project Planning and Cost Estimation **19**
- 4.1 Project Development Methodology . . . 19
- 4.2 Project Schedule and Gantt Chart . . . 20
- 4.3 Team Roles and Responsibilities . . . 21
- 4.4 Software Size Estimation . . . 21
  - 4.4.1 KLOC-Based Estimation . . . 21
- 4.5 Effort Estimation Using COCOMO . . . 22
  - 4.5.1 Basic COCOMO Model . . . 22
  - 4.5.2 Calculation . . . 22
- 4.6 Human Resource Cost . . . 23
- 4.7 Hardware, Software and Cloud Infrastructure Cost . . . 24
- 4.8 Testing, Deployment and Maintenance Cost . . . 24

4.9 Total Estimated Project Cost . . . 25
4.10 Assumptions and Cost Justification . . . 25
4.11 Risk Assessment and Mitigation . . . 26

5 Project Implementation **28**
5.1 Code Snippets . . . 28
5.1.1 Main Application Navigation . . . 28
5.1.2 Underwater Image Validation . . . 29
5.1.3 Image Analysis Workflow . . . 30
5.1.4 Underwater Image Enhancement . . . 31
5.1.5 Video Detection Workflow . . . 33
5.1.6 Inference Service . . . 34
5.2 Steps to Access the System . . . 35
5.3 System Implementation . . . 35
5.3.1 Oceanic Intelligence Dashboard . . . 36
5.3.2 Underwater Image Detection Interface . . . 36
5.3.3 Underwater Image Enhancement Interface . . . 37
5.3.4 Pollution Analysis Interface . . . 38
5.3.5 Analytics and Survey Telemetry . . . 38
5.3.6 Implementation Summary . . . 39
5.4 Timeline Sem VII . . . 39

6 Testing **41**
6.1 Software Testing . . . 41
6.2 Types of Testing . . . 41
6.2.1 Functional Testing . . . 41
6.2.2 System Testing . . . 42
6.3 Testing Results and Findings . . . 42
6.4 Sample Test Cases . . . 42
6.5 Conclusion on Testing . . . 43

7 Result and Discussions **44**
7.1 Introduction . . . 44
7.2 Evaluation Metrics . . . 44
7.3 Training and Validation Performance . . . 44
7.4 Precision, Recall, and F1-Score Analysis . . . 46
7.5 Cross-Domain Model Performance Comparison . . . 46
7.6 Confusion Matrix Analysis . . . 46
7.7 Precision–Recall Curve Analysis . . . 48
7.8 Class Distribution and Environmental Threat Analysis . . . 51
7.9 Resolution and SAHI Ablation Study . . . 52
7.10 Overall Discussion and Limitations . . . 52
7.11 Chapter Summary . . . 52

8 Conclusion **53**

9 Future Scope **54**

Bibliography **55**

List of Figures

3.1 Overall Architecture of the Proposed Marine Debris Detection System using YOLOv11 . . . 15
3.2 UML Class Diagram of the Proposed NirmalSagar System . . . 16
3.3 Activity Diagram of the Proposed NirmalSagar System . . . 17
3.4 Use Case Diagram of the Proposed NirmalSagar System . . . 18
3.5 Sequence Diagram of the Proposed NirmalSagar System . . . 18

5.1 Main React Application and View Management . . . 29
5.2 Underwater Image Upload and Validation . . . 30
5.3 Image Analysis Workflow and Result Handling . . . 31
5.4 Canvas-Based Underwater Image Enhancement . . . 32
5.5 Video Detection and Result Handling . . . 33
5.6 Image Inference Service and Backend Integration . . . 34
5.7 NirmalSagar Oceanic Intelligence Dashboard . . . 36
5.8 Underwater Image Detection and Detection Summary . . . 37
5.9 Underwater Image Enhancement Interface . . . 37
5.10 Pollution Analysis and Ecological Risk Assessment . . . 38
5.11 Analytics Dashboard and Session Telemetry . . . 39
5.12 Gantt Chart of the NirmalSagar Major Project . . . 40

7.1 Training and validation loss convergence across the model optimisation process. 45
7.2 Progression of detection and segmentation metrics during training. . . . 45
7.3 Precision, recall, and F1-score progression across training epochs. . . . 46
7.4 Cross-domain performance comparison across the TrashCan, SeaClear, and JAMSTEC datasets. . . . 47
7.5 Confusion matrix for the evaluated underwater debris detection model. . . . 47
7.6 Normalised confusion matrix for the evaluated model. . . . 48
7.7 Bounding-box precision–recall curve for underwater debris detection. . . . 48
7.8 Segmentation-mask precision–recall curve. . . . 49
7.9 Bounding-box precision across confidence thresholds. . . . 49
7.10 Bounding-box recall across confidence thresholds. . . . 50
7.11 Segmentation-mask precision across confidence thresholds. . . . 50
7.12 Segmentation-mask recall across confidence thresholds. . . . 51
7.13 Taxonomy of debris categories and associated environmental threat descriptions. 51
7.14 Comparison of image-resolution settings and SAHI for debris detection. . . . 52

List of Tables

2.1 Summary of Literature Review on Underwater Marine Debris Detection . . . 10

4.1 Planned Project Schedule for NirmalSagar 20
4.2 Team Roles and Responsibilities 21
4.3 Preliminary Software Size Estimation 22
4.4 Basic COCOMO Effort Estimation 23
4.5 Human Resource Cost for the Academic Project 23
4.6 Hardware, Software and Infrastructure Cost 24
4.7 Testing, Deployment and Maintenance Cost 25
4.8 Summary of Estimated Direct Project Cost 25
4.9 Risk Assessment and Mitigation for NirmalSagar 26

6.1 Representative Test Cases for NirmalSagar 43

List of Abbreviations

YOLO	: You Only Look Once
CNN	: Convolutional Neural Network
AI	: Artificial Intelligence
ML	: Machine Learning
mAP	: mean Average Precision
IoU	: Intersection over Union
NMS	: Non-Maximum Suppression
ROV	: Remotely Operated Vehicle
AUV	: Autonomous Underwater Vehicle
FPS	: Frames Per Second
mAP@0.50	: Mean Average Precision at IoU 0.50
mAP@0.50:0.95	: Mean Average Precision at IoU 0.50:0.95
P	: Precision
R	: Recall
F1	: F1-score

Chapter 1

Introduction

Marine pollution has become an important environmental concern due to the increasing amount of waste entering oceans, coastal regions, and underwater environments. Marine debris includes plastics, fishing nets, bottles, containers, and other forms of waste that can remain in aquatic environments for long periods. Such debris can affect marine ecosystems and may pose risks to marine organisms through ingestion, entanglement, and habitat disturbance. Monitoring and identifying marine debris is therefore important for understanding underwater pollution and supporting effective cleanup and conservation activities.

Underwater marine debris detection is a challenging computer vision problem because underwater images often differ significantly from images captured in normal terrestrial environments. Factors such as turbidity, low visibility, low light, lighting variations, and complex underwater backgrounds can make debris difficult to distinguish from surrounding objects. Changes in water conditions can also affect the appearance, colour, contrast, and visibility of marine litter. As a result, conventional image-based detection methods may experience difficulties when applied to underwater environments with different visual characteristics.

Manual inspection of underwater environments is another major challenge. Identifying and recording marine debris through human inspection requires considerable time and effort, particularly when large underwater areas have to be monitored. The process can also become difficult when visibility is poor or when debris is partially hidden within complex underwater backgrounds. These limitations create a need for automated computer vision and artificial intelligence techniques that can assist in detecting and localizing marine debris from underwater images.

The proposed project, NirmalSagar: Underwater Marine Litter & Plastic Detection, focuses on developing an AI-based approach for automated detection and localization of underwater marine debris. The system combines image enhancement and YOLO-based object detection to improve the identification of marine litter under challenging underwater conditions. The project also considers the problem of domain shift, where a model trained using one underwater dataset or environment may not perform equally well when applied to another environment with different visual characteristics.

To address this challenge, the project focuses on domain adaptation and generalisation strategies using techniques such as synthetic augmentation, style transfer, and adversarial domain adaptation. These approaches are intended to improve the ability of the trained model to generalise from existing datasets such as TrashCan and JAMSTEC to unseen and

murkier Indian coastal and seafloor conditions. The performance of the detection system is evaluated across heterogeneous underwater marine-debris datasets using standard evaluation measures such as mAP, IoU, and F1-score.

In addition to object detection, the project considers the estimation of debris density from detected objects. Per-frame detections can be aggregated to obtain an area-level pollution score, which can provide useful information about the concentration of marine debris in a particular area. Such an approach can support the prioritisation of cleanup activities along Indian coastlines. Therefore, the overall project focuses not only on detecting individual marine debris objects but also on developing a broader approach for automated underwater pollution monitoring.

1.1 Motivation

The motivation for this project comes from the increasing need for efficient monitoring of marine pollution and the limitations associated with manual underwater inspection. Marine debris such as plastics, fishing nets, bottles, and other waste can remain in underwater environments and may affect marine ecosystems and marine life. Identifying such debris at an early stage can support better monitoring and help in planning appropriate cleanup activities.

Underwater environments present several visual challenges for automated detection. Turbidity can reduce visibility, while low light and lighting variations can change the appearance of objects in underwater images. Complex backgrounds can further make it difficult to distinguish marine debris from natural underwater objects. These challenges make underwater marine debris detection different from conventional object detection performed on clear images.

Another motivation is the time and human effort required for manual monitoring. When underwater environments need to be inspected over large areas or across a large number of images, manually identifying individual debris objects becomes difficult and time-consuming. An automated AI-based system can assist in processing underwater images and identifying possible debris objects more efficiently.

The project is also motivated by the need for models that can work beyond the conditions represented in their training datasets. A model trained on a particular dataset may encounter difficulties when applied to an unseen underwater environment having different levels of turbidity, lighting, background complexity, or visual appearance. Therefore, domain adaptation and generalisation are important considerations for developing a detection system that can operate under different underwater conditions.

The proposed NirmalSagar system aims to address these challenges by combining image enhancement, deep learning, computer vision, and YOLO-based object detection. The project further investigates generalisation across multiple heterogeneous datasets and considers murkier Indian coastal and seafloor conditions. This can contribute towards faster and more consistent marine pollution monitoring while reducing dependence on completely manual inspection.

1.2 Problem Statement

The detection of marine debris in underwater environments is difficult because underwater images are affected by several environmental and visual factors. Turbidity, low visibility, low light, lighting variations, and complex backgrounds can significantly change the appearance of objects in an image. Marine debris may also be partially obscured or difficult to distinguish from surrounding underwater structures.

Marine litter such as plastics, fishing nets, bottles, and other waste can pose a threat to marine life and underwater ecosystems. However, identifying these objects manually from underwater images requires significant human effort and can become time-consuming when a large number of images or large underwater areas need to be inspected. This creates a requirement for automated techniques that can assist in identifying and localizing marine debris efficiently.

A further challenge is the variation between different underwater environments and datasets. A detection model trained using images collected under one set of environmental conditions may not provide the same level of performance when applied to images from another location. Differences in water clarity, lighting, background characteristics, and object appearance can result in domain-shift degradation.

Therefore, the problem addressed by this project is the development of an AI-based system for automatic detection and localization of submerged marine debris from underwater images while considering challenging underwater conditions and differences between datasets. The project focuses on combining image enhancement with YOLO-based detection and evaluating domain adaptation and generalisation strategies to improve performance across heterogeneous underwater environments.

1.3 Objectives

The objectives of the proposed NirmalSagar project are derived from the challenges associated with underwater marine debris detection, dataset variation, and marine pollution monitoring.

- To develop an underwater debris detection pipeline that combines image enhancement with YOLO for accurate detection and classification of marine litter from underwater images.
- To investigate the effect of challenging underwater conditions such as turbidity, low visibility, low light, lighting variations, and complex backgrounds on marine debris detection.
- To develop and evaluate domain adaptation and generalisation strategies including synthetic augmentation, style transfer, and adversarial domain adaptation so that a model trained on one dataset or site, such as TrashCan and JAMSTEC, can generalise to unseen and murkier Indian coastal and seafloor conditions.

- To benchmark the performance of the detection system across multiple heterogeneous datasets including TrashCan 1.0, Underwater Marine Debris in Shallow Waters, and JAMSTEC Deep-sea Debris Database.
- To quantify domain-shift degradation using standard evaluation metrics such as mean Average Precision (mAP), Intersection over Union (IoU), and F1-score.
- To develop a debris-density estimation approach that aggregates per-frame detections into an area-level pollution score for representing the concentration of detected marine debris.
- To support the prioritisation of cleanup activities along Indian coastlines by providing an area-level indication of marine debris density.

1.4 Scope

The scope of the NirmalSagar project covers the automated analysis of underwater images for detecting and localizing marine debris. The project focuses on computer vision and deep learning techniques that can assist in identifying marine litter under challenging underwater conditions.

- **Automated Marine Debris Detection:**

  The system focuses on automated detection and localization of submerged marine debris from underwater images using deep learning and computer vision techniques.

- **Underwater Image Analysis:**

  The project considers underwater images affected by challenging conditions such as low visibility, turbidity, low light, lighting variations, and complex backgrounds.

- **Image Enhancement:**

  Image enhancement techniques are considered as part of the detection pipeline to improve the visual characteristics of underwater images before applying object detection.

- **YOLO-Based Object Detection:**

  The project uses YOLO-based object detection for identifying and localizing marine litter within underwater images.

- **Domain Adaptation and Generalisation:**

  The project considers synthetic augmentation, style transfer, and adversarial domain adaptation strategies to improve model generalisation across different underwater environments and datasets.

- **Multi-Dataset Evaluation:**

  The detection system is evaluated across heterogeneous marine-debris datasets including TrashCan 1.0, Underwater Marine Debris in Shallow Waters, and JAMSTEC Deep-sea Debris Database.

- **Performance Evaluation:**

  The project evaluates detection performance using standard metrics such as mAP, IoU, and F1-score, while also considering the degradation caused by domain shifts between datasets and environments.

- **Debris-Density Estimation:**

  The project includes aggregation of per-frame detections to estimate an area-level pollution score representing debris density.

- **Indian Coastal and Seafloor Conditions:**

  The project specifically considers the generalisation of detection models towards unseen and murkier Indian coastal and seafloor conditions.

- **Marine Pollution Monitoring:**

  The resulting debris-density information can support marine pollution monitoring and help in prioritising cleanup activities along Indian coastlines.

Chapter 2

Literature Review

In recent years, there has been growing interest in the application of artificial intelligence, deep learning, and computer vision for underwater object and marine debris detection. Marine debris detection is challenging because underwater images are affected by factors such as turbidity, low visibility, lighting variations, complex backgrounds, and the presence of small or camouflaged objects. Several studies have explored different object detection architectures, underwater marine-debris datasets, image enhancement techniques, and domain generalisation approaches. This literature review summarizes important developments in underwater object and marine-debris detection and identifies the research gaps that motivate the proposed NirmalSagar system.

2.1 Comparative Analysis of Recent Studies

Different approaches for underwater object and marine-debris detection have been reviewed in recent studies. Traditional hand-crafted features as well as deep learning-based object detection architectures have been explored for identifying underwater objects. Commonly investigated architectures include SSD, YOLO-based models, FPN, PANet, BiFPN, Faster R-CNN, and other deep learning approaches. Although these methods have improved automated detection, challenges related to small objects, camouflage, turbidity, dataset imbalance, and generalisation across underwater environments remain.

The development of deep learning-based marine debris detection has provided improved capabilities for automated detection and monitoring. Survey studies have highlighted the increasing use of deep learning architectures for underwater object detection and have also identified limitations in the diversity and generalisability of available datasets. Existing datasets and studies differ in terms of underwater environments, camera conditions, object categories, and image quality, making it difficult to achieve reliable performance across different locations.

Several studies have specifically focused on marine-debris datasets and detection models. Duraš et al. [5] introduced the SeaClear multi-domain dataset containing 8,610 images and 40 classes and evaluated baseline detection using Faster R-CNN and YOLOv6. The study also investigated image enhancement and highlighted challenges related to imbalanced categories and cross-site and cross-camera generalisation.

Jiang et al. [6] proposed an improved YOLOv8m model using a C2f-CloFormer backbone, C2f-SCConv neck modules, and WIoU v3 regression loss for marine-debris identification and classification. Although the approach improves the detection model, the study reports a trade-off between accuracy and inference speed, with lower inference FPS than the baseline. Recognition performance also decreases for some debris categories.

Zhang et al. [7] proposed an improved YOLOv11 model using a FasterNet backbone with C2f-Faster and SE channel-attention modules for lightweight underwater marine-debris detection. The study focuses on improving detection efficiency, although large-scale variation of underwater targets and near/far distortion remain unresolved challenges.

Khriss et al. [8] comparatively evaluated Faster R-CNN, SSD, YOLOv8, and YOLOv9 for underwater plastic-debris detection and monitoring. The study highlights challenges in detecting rare or less-common objects and reports that cluttered and noisy underwater scenes continue to affect detection performance.

Zhang et al. [9] proposed UDD-YOLO, which enhances a YOLOv12n baseline using a Cold Diffusion preprocessor, an AMC2f multi-scale feature module, and a Unified-IoU loss. The approach addresses underwater image quality and marine-debris detection; however, the diffusion preprocessing introduces additional processing latency on edge hardware.

The reviewed survey studies also provide important observations regarding the broader research area. Er et al. [1] reviewed deep learning architectures and popular datasets for underwater marine object detection and highlighted the absence of a universally accepted benchmark dataset, inter-class similarity, camouflage, and dataset imbalance. Jian et al. [2] surveyed traditional and deep learning-based underwater object detection methods and identified weaknesses in detecting small and camouflaged targets along with limitations in dataset diversity and generalisability.

Moorton et al. [3] reviewed deep learning applications for tracking and detecting marine debris and conducted an original YOLOv5 experiment using a small custom dataset. The study highlighted limitations associated with small datasets, limited public availability of data, and the exclusion of micro- and nano-debris. Moorton et al. [4] further conducted a PRISMA-ScR scoping review of object detection methods for macro marine debris and identified limited diversity in benchmark datasets and inconsistent reporting of metrics and data.

2.2 Research Gap

The literature review highlights that although considerable progress has been made in applying deep learning to underwater object and marine-debris detection, several limitations remain. Existing annotated underwater marine-debris datasets have limited diversity, while variations in underwater environments, cameras, and sites make generalisation difficult. Poor image quality caused by turbidity, low light, and colour distortion further affects the visibility of marine debris. Small, rare, and underrepresented debris categories also remain challenging because of class imbalance and the difficulty of detecting small or camouflaged objects.

Another important limitation is the limited generalisation of models across different un-

derwater environments. Models trained on a particular dataset or site may not perform consistently when applied to unseen and murkier underwater conditions. Differences in image quality, environmental conditions, camera characteristics, and debris categories contribute to domain-shift degradation.

Therefore, the proposed work focuses on using multiple publicly available underwater marine-debris datasets to provide greater variation in debris types and underwater conditions. Underwater image enhancement and preprocessing are considered to improve the visibility of debris features before detection. Training and evaluation on diverse underwater imagery are intended to improve robustness beyond a single dataset or environment. YOLO-based detection with suitable data augmentation and class-balancing techniques is considered to improve the detection of small, difficult, and underrepresented debris categories.

The research gap identified from the reviewed studies therefore motivates the development of an AI-based marine-debris detection system that combines underwater image processing, deep learning-based object detection, and evaluation across diverse underwater datasets. The proposed NirmalSagar system is aimed at addressing the challenges of automated marine-debris detection and localisation under difficult underwater conditions.

Ref.	**Author(s) and Year**	**Key Contributions**	**Limitations / Gaps Identified**
[1]	Er, M.J., Chen, J., Zhang, Y., Gao, W. (2023)	Reviewed deep learning-based underwater marine object detection architectures including SSD, YOLOv3, FPN, PANet, BiFPN, and Faster R-CNN and organized the research challenges into four categories.	No universally accepted benchmark dataset exists. Inter-class similarity and camouflage remain unresolved, while reviewed datasets show imbalance and small-scale objects.
[2]	Jian, M., Yang, N., Tao, C., Zhi, H., Luo, H. (2024)	Surveyed 14 traditional hand-crafted-feature methods and 34 deep learning methods across eight categories and reviewed representative underwater object-detection datasets.	Deep learning models remain weak on small and camouflaged targets. Turbidity and refraction remain challenging, while existing datasets lack diversity and generalisability.
[3]	Moorton, Z., Kurt, Z., Woo, W.L. (2024)	Reviewed 28 deep learning studies on marine-debris detection and conducted an original YOLOv5 experiment using a small custom litter-versus-animal dataset.	The custom dataset was too small for reliable results, with maximum reported accuracy of about 63%. Most surveyed studies did not share data publicly and micro- and nano-debris were excluded.
[4]	Moorton, Z., Mistry, K., Strachan, R., Hu, S. (2025)	Conducted a PRISMA-ScR scoping review of 25 papers with comparative analysis of models, datasets, and hardware and proposed a reporting-standardisation framework.	Benchmark datasets lack diversity and are largely based on JAMSTEC-derived deep-sea imagery. Inconsistent metric and data reporting limits comparison between studies.

Ref.	Author(s) and Year	Key Contributions	Limitations / Gaps Identified
[5]	Đuraš, A., Wolf, B.J., Ilioudi, A., Palunko, I., De Schutter, B. (2024)	Introduced the SeaClear multi-domain dataset containing 8,610 images and 40 classes and evaluated baseline detection using Faster R-CNN and YOLOv6 along with image-enhancement analysis.	Category distribution is imbalanced across sites and cameras. Image enhancement did not improve cross-site and cross-camera generalisation, and some sites contain very few images.
[6]	Jiang, W., Yang, L., Bu, Y. (2024)	Proposed an improved YOLOv8m using a C2f-CloFormer backbone, C2f-SCConv neck modules, and WIoU v3 regression loss for marine-debris identification and classification.	Lower inference FPS than the baseline creates an accuracy-speed trade-off. The model is not fully lightweight and recognition decreases for some classes such as can, branch, wreckage, and tarp.
[7]	Zhang, L., Yang, J., Yu, H. (2025)	Improved YOLOv11 using a FasterNet backbone with C2f-Faster and SE channel-attention modules for lightweight underwater marine-debris detection.	No explicit limitations section was reported. Large-scale variation of underwater targets and near/far distortion remain unresolved challenges.
[8]	Khriss, A., Kerkour Elmiad, A., Badaoui, M., Barkaoui, A.-E., Zarhloule, Y. (2024)	Compared Faster R-CNN, SSD, YOLOv8, and YOLOv9 for underwater plastic-debris detection and monitoring.	Poor detection of rare or less-common objects and varying model stability remain challenges. Cluttered and noisy underwater scenes also affect detection.
[9]	Zhang, C., Zhang, W., Liang, C., Qiao, Q., Tao, J. (2025)	Proposed UDD-YOLO using a YOLOv12n baseline enhanced with a Cold Diffusion preprocessor, AMC2f multi-scale feature module, and Unified-IoU loss.	Diffusion preprocessing introduces additional per-frame latency on edge hardware, with the reported FPS decreasing from 79.1 to 65.9 on Jetson Xavier NX.

Table 2.1: Summary of Literature Review on Underwater Marine Debris Detection

Chapter 3

Project Design

3.1 Existing System

The existing approaches for underwater marine debris monitoring mainly depend on manual inspection and image-based detection methods developed for specific underwater environments. Manual inspection involves examining underwater environments and identifying marine debris such as plastics, fishing nets, bottles, and other waste through human observation. Although manual monitoring can provide useful information about underwater pollution, it requires considerable time and human effort, especially when large underwater areas or a large number of images need to be inspected.

Underwater image-based detection systems have also been developed using computer vision and deep learning techniques. However, the performance of such systems can be affected by the visual characteristics of underwater environments. Factors such as turbidity, low visibility, low light, lighting variations, and complex backgrounds can make marine debris difficult to identify. Objects may appear different depending on water conditions, illumination, and the surrounding underwater environment.

Another limitation of existing detection approaches is their dependency on the dataset and environment used during training. A model trained on images collected from one location or dataset may not provide the same performance when applied to an unseen environment. Differences in water clarity, lighting conditions, background characteristics, and object appearance can result in domain-shift degradation.

Existing approaches may therefore perform well under the conditions represented in their training datasets but may face difficulties when applied to different or murkier underwater environments. This is particularly important for marine debris monitoring in Indian coastal and seafloor conditions, where underwater visual characteristics may differ from those present in existing benchmark datasets.

Thus, the existing system can be summarized as follows:

- Manual underwater inspection requires significant time and human effort for identifying marine debris.
- Underwater image quality is affected by turbidity, low visibility, low light, lighting variations, and complex backgrounds.
- Conventional detection approaches may have difficulty distinguishing marine debris from

surrounding underwater objects.

- Models trained on a particular dataset or environment may experience performance degradation when applied to unseen underwater conditions.
- Existing approaches require improved generalisation strategies to handle differences between heterogeneous underwater datasets and real-world environments.

3.2 Proposed System

The proposed system, NirmalSagar: Underwater Marine Litter & Plastic Detection, is designed as an AI-based approach for automated detection and localization of marine debris from underwater images. The system focuses on addressing the challenges associated with underwater visual conditions and differences between datasets.

The proposed detection pipeline combines underwater image enhancement with YOLO-based object detection. Image enhancement is used as part of the processing pipeline to improve the visual characteristics of underwater images affected by conditions such as low visibility, turbidity, low light, and lighting variations. The enhanced images are then processed using a YOLO-based detection approach to identify and localize marine debris.

A major component of the proposed system is the consideration of domain adaptation and generalisation. A model trained using one underwater dataset or site may not perform equally well when applied to another environment. To address this issue, the project considers strategies including synthetic augmentation, style transfer, and adversarial domain adaptation. These approaches are intended to improve the ability of the detection model to generalise from existing datasets such as TrashCan and JAMSTEC to unseen and murkier Indian coastal and seafloor conditions.

The proposed system also evaluates the detection approach across multiple heterogeneous marine-debris datasets. The datasets considered for benchmarking include TrashCan 1.0, Underwater Marine Debris in Shallow Waters, and the JAMSTEC Deep-sea Debris Database. Performance is evaluated using standard metrics such as mean Average Precision (mAP), Intersection over Union (IoU), and F1-score. These measures help quantify the effectiveness of the detection system and the degradation caused by domain shifts.

In addition to individual object detection, the proposed system includes a debris-density estimation approach. Per-frame detections can be aggregated to estimate an area-level pollution score representing the concentration of marine debris in a particular area. This information can support marine pollution monitoring and help in prioritising cleanup activities along Indian coastlines.

The overall proposed system therefore focuses on combining image enhancement, deep learning, YOLO-based object detection, domain adaptation and generalisation, multi-dataset evaluation, and debris-density estimation into a unified approach for underwater marine debris monitoring.

The major components of the proposed system are:

- **Underwater Image Input:** Underwater images are provided as input to the detection pipeline for analysis of marine debris.
- **Image Enhancement:** Image enhancement is applied to improve the visual characteristics of underwater images affected by challenging environmental conditions.
- **YOLO-Based Detection:** A YOLO-based object detection approach is used to identify and localize marine debris within underwater images.

- **Domain Adaptation and Generalisation:** Synthetic augmentation, style transfer, and adversarial domain adaptation strategies are considered to improve model generalisation across different underwater environments.
- **Multi-Dataset Benchmarking:** The detection system is evaluated using TrashCan 1.0, Underwater Marine Debris in Shallow Waters, and the JAMSTEC Deep-sea Debris Database.
- **Performance Evaluation:** The system is evaluated using mAP, IoU, and F1-score to measure detection performance and quantify domain-shift degradation.
- **Debris-Density Estimation:** Per-frame detections are aggregated to estimate an area-level pollution score representing the density of marine debris.
- **Marine Pollution Monitoring:** The estimated debris density can support the prioritisation of cleanup activities along Indian coastlines.

3.3 Critical Components of System Architecture

The proposed NirmalSagar system consists of a sequence of interconnected components that together form the complete underwater marine debris detection pipeline. The architecture begins with the acquisition of underwater images or video sequences and proceeds through data preparation, YOLOv11 model training, inference, and generation of detection results. The major components of the proposed architecture are described below.

3.3.1 Data Acquisition

The first stage of the proposed system is data acquisition. Underwater images and video sequences are collected from underwater environments using suitable underwater cameras and ROV/AUV-based systems. The collected data can represent different underwater conditions, including variations in lighting, turbidity, and depth.

The acquired images and video sequences provide the input data required for subsequent preprocessing, annotation, model training, and evaluation. A diverse collection of underwater data is important because marine debris can appear differently under different environmental conditions.

3.3.2 Data Preparation

The collected underwater data undergoes several preparation steps before being provided to the detection model. The data preparation stage consists of preprocessing, annotation, data augmentation, and dataset splitting.

Preprocessing: The underwater images are prepared by performing operations such as resizing, denoising, and colour and contrast enhancement. These operations help improve the visual characteristics of the images before they are used by the detection model.

Annotation: The marine debris present in the images is annotated using bounding boxes and corresponding class labels. These annotations provide the information required by the object detection model to learn the location and category of marine debris.

Data Augmentation: Data augmentation is applied to increase the diversity of the training data. The architecture considers augmentation operations such as flipping, rotation, Mosaic,

MixUp, and colour jitter. These transformations help expose the model to variations in the appearance and orientation of underwater objects.

Dataset Split: The prepared dataset is divided into training, validation, and testing data. Training data is used for learning the model parameters, validation data is used during model development, and test data is used to evaluate the trained model on previously unseen data.

3.3.3 YOLOv11 Model Training

The prepared training data is provided to the YOLOv11 model during the training stage. The YOLOv11 architecture consists of three major components: the backbone, neck, and detection head.

Backbone: The backbone performs feature extraction from the input images. It learns visual features that help distinguish marine debris from the surrounding underwater environment.

Neck: The neck performs feature fusion by combining information from different feature levels. This helps the detection model use both detailed and higher-level visual information during object detection.

Detection Head: The detection head produces the final detection-related outputs. These include class probabilities, bounding box regression, and objectness scores for the detected objects.

During training, the model uses a training strategy involving loss calculation, optimization, learning-rate scheduling, hyperparameter tuning, early stopping, and model checkpointing. The model parameters are updated through backpropagation and weight updates based on the training process.

3.3.4 Trained Model

After completion of the training process, the learned YOLOv11 model is saved as the trained model. The trained model contains the learned parameters required to detect marine debris in new underwater images or video sequences.

The trained model can subsequently be used during the inference stage to process previously unseen underwater data. The separation between training and inference allows the trained model to be evaluated independently on test data and new underwater inputs.

3.3.5 Inference and Detection

The inference stage uses the trained YOLOv11 model to detect marine debris in new underwater images or video sequences. The input undergoes preprocessing similar to the training pipeline, including resizing, enhancement, and normalization.

The preprocessed input is then passed through the YOLOv11 model during the forward pass. Non-Maximum Suppression (NMS) is applied to remove overlapping detections and retain the relevant bounding boxes.

The detection stage produces information including the class label, bounding box coordinates, and confidence score for each detected marine debris object. These results provide the basis for further analysis and visualization.

3.3.6 Output and Applications

The final stage of the architecture produces annotated detection results. Detected marine debris objects are represented using bounding boxes along with their corresponding class labels and confidence scores.

The resulting information can be used for underwater pollution monitoring and marine conservation activities. The detected data can also support data analysis and reporting and provide information that can assist authorities in making decisions related to marine pollution monitoring and cleanup activities.

The complete architecture therefore establishes a flow from underwater data acquisition to data preparation, YOLOv11 training, model generation, inference, and final detection output. This integrated pipeline provides the foundation for automated underwater marine debris detection using deep learning.

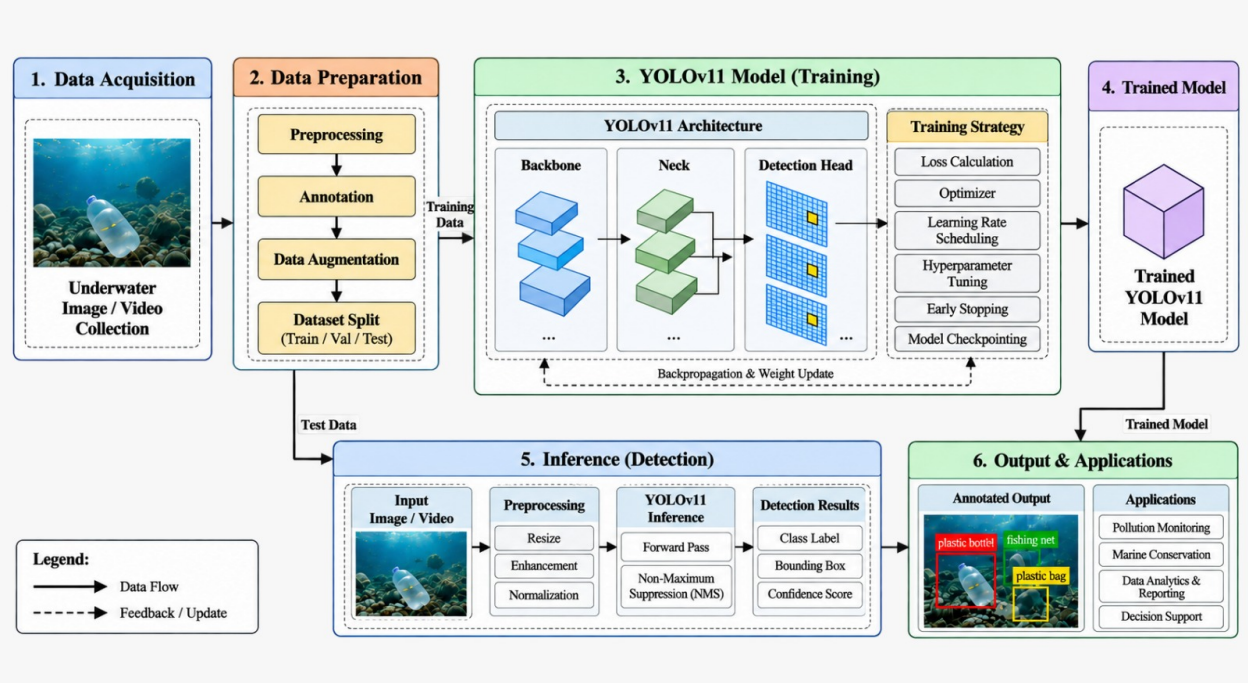


Figure 3.1: Overall Architecture of the Proposed Marine Debris Detection System using YOLOv11

3.4 System Diagrams

System diagrams provide a visual representation of the structure, behaviour, and interaction of the proposed NirmalSagar system. They help in understanding the different components of the system and the flow of information between the user and the detection modules. The major diagrams used for the proposed system are the UML Diagram, Activity Diagram, Use Case Diagram, and Sequence Diagram.

3.4.1 UML Diagram

The UML diagram represents the major classes and components involved in the NirmalSagar system. It shows the structural relationship between the user, preprocessing module, dataset, YOLOv11 model, detection result, image enhancement, and debris analysis components.

The *User* interacts with the system by providing underwater images or videos and viewing the generated detection results. The *Preprocessor* performs the required preprocessing operations before the data is passed to the YOLOv11 model. The *Dataset* represents the collection of underwater marine debris images used for training and evaluation.

The *YOLOv11 Model* performs marine debris detection on the processed input. The detection results are then passed to the *Debris Analyzer*, which can be used to analyse the detected debris and estimate pollution-related information. The *Detection Result* stores the output generated by the detection process. The relationships between these classes describe the flow of data through the proposed system.

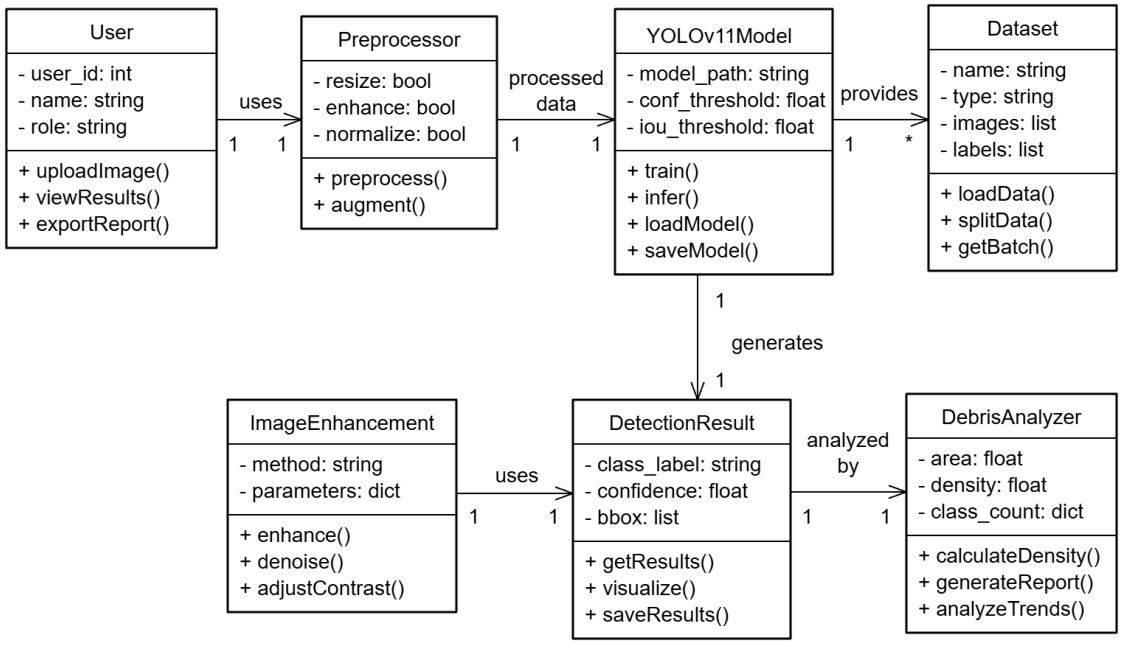


Figure 3.2: UML Class Diagram of the Proposed NirmalSagar System

3.4.2 Activity Diagram

The activity diagram represents the workflow of the NirmalSagar system from input acquisition to the generation of final detection results. The process begins when an underwater image or video is provided to the system. The input is then passed through preprocessing before further processing.

The diagram represents two major workflows: model training and model inference. During training, images are annotated, augmented, divided into appropriate dataset splits, and used to train the YOLOv11 model. The trained model is evaluated and saved for future inference.

During inference, the trained model is loaded and a new underwater image is preprocessed. YOLOv11 performs detection followed by Non-Maximum Suppression (NMS) to remove overlapping detections. The detection results are then generated, debris density can be estimated, and the final results are displayed or stored.

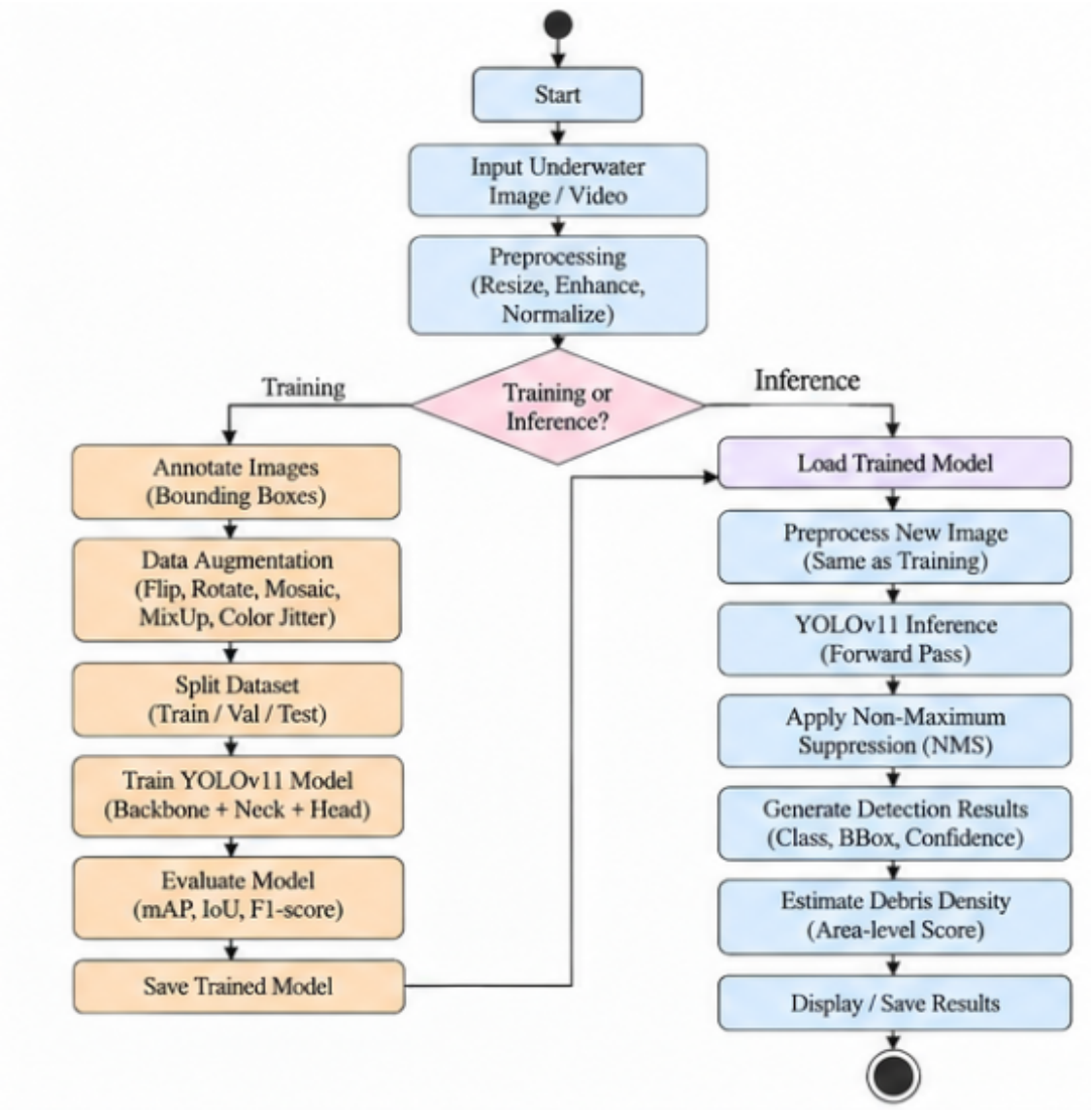


Figure 3.3: Activity Diagram of the Proposed NirmalSagar System

3.4.3 Use Case Diagram

The use case diagram illustrates the interaction between the users and the NirmalSagar system. It identifies the major functions provided by the system and the actors involved in those functions.

The primary user can upload underwater images or videos, view detection results, view debris density reports, and export the generated results. The system performs image preprocessing and marine debris detection using the trained YOLOv11 model. It also supports debris density estimation and generation of pollution-related reports.

The administrator or data manager is responsible for activities related to dataset management and annotation. The system also includes functions for model validation and evaluation. These use cases provide an overall view of how different users interact with the proposed marine debris detection system.

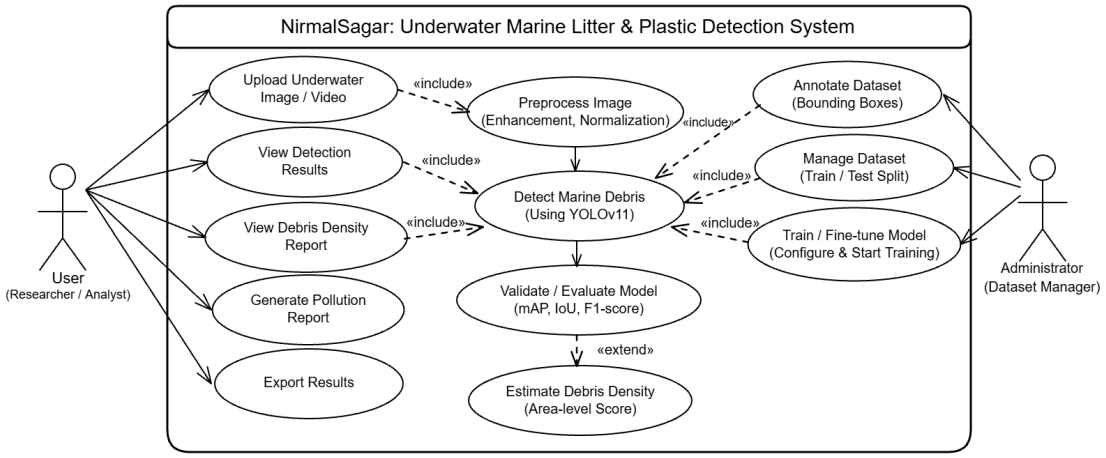


Figure 3.4: Use Case Diagram of the Proposed NirmalSagar System

3.4.4 Sequence Diagram

The sequence diagram represents the chronological interaction between the different components of the NirmalSagar system during the detection process. It shows how information is transferred from the user to the system and subsequently processed by the different modules.

The process starts when the user uploads an underwater image or video through the system interface. The input is passed to the preprocessing module, where the required preprocessing operations are performed. The processed input is then sent to the trained YOLOv11 model for inference.

After detection, the detection module processes the output and generates the detected marine debris information. The Debris Analyzer can then calculate the required debris density information. Finally, the result visualizer displays the detection results to the user, who can view or export the generated output.

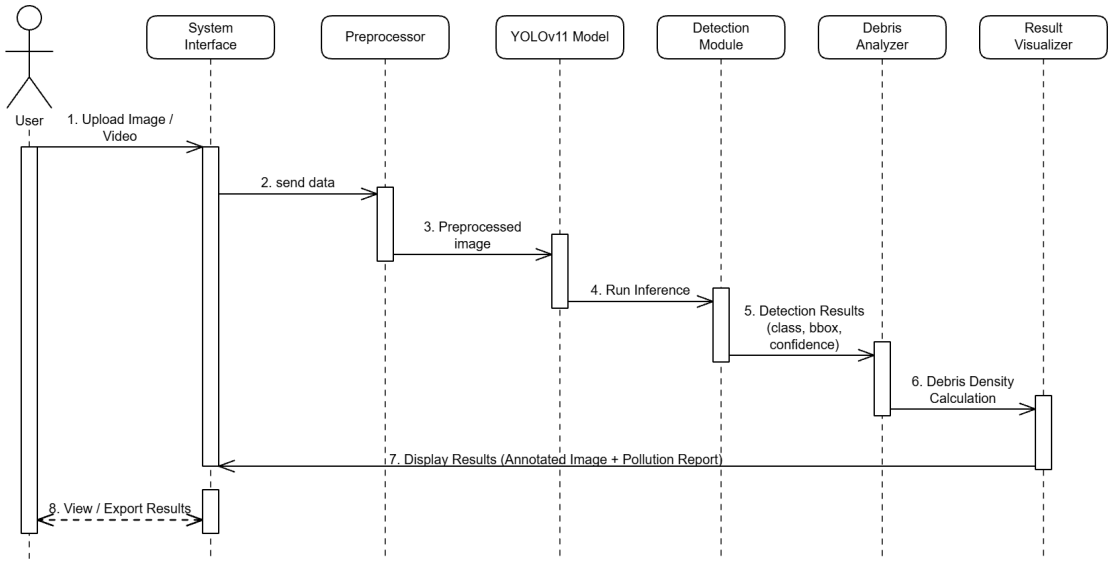


Figure 3.5: Sequence Diagram of the Proposed NirmalSagar System

Chapter 4

Project Planning and Cost Estimation

4.1 Project Development Methodology

The NirmalSagar project follows an iterative and incremental development methodology. The system is developed in multiple stages, where each stage focuses on a specific component of the underwater marine debris detection pipeline. This approach allows individual modules to be developed, integrated, tested, and improved progressively.

The major stages of the development methodology are described below:

1. **Requirement Analysis:** Identify the project objectives, functional requirements, datasets, and expected system outputs for underwater marine debris detection.
2. **Dataset Acquisition and Preprocessing:** Collect publicly available underwater marine debris datasets, including TrashCan 1.0, the Underwater Marine Debris in Shallow Waters dataset, and the JAMSTEC Deep-Sea Debris Database. Prepare the images and annotations for model development.
3. **Image Enhancement:** Apply suitable underwater image enhancement and preprocessing techniques to improve image visibility and investigate their effect on debris detection.
4. **Model Development:** Develop and evaluate object detection and instance segmentation models, including YOLO-based approaches, for identifying and localising marine debris.
5. **Model Evaluation:** Evaluate model performance using metrics such as precision, recall, mean Average Precision (mAP), and Intersection over Union (IoU). Compare results across different datasets to examine cross-domain generalisation.
6. **Application Integration:** Integrate the trained model with the application interface to support image and video analysis, visualisation of detection results, and pollution assessment.
7. **Testing and Refinement:** Test the integrated application, identify issues, and refine the relevant components based on observed results.
8. **Final Documentation and Deployment:** Prepare the project documentation and presentation, and organise the application and its dependencies for demonstration or deployment.

The iterative nature of this methodology allows feedback from model evaluation and system testing to guide improvements in subsequent development stages.

4.2 Project Schedule and Gantt Chart

The project schedule defines the major activities involved in the development of NirmalSagar and their planned sequence. The schedule is based on the project timeline described in the project proposal. It includes requirement analysis, dataset preparation, model development, evaluation, application integration, testing, and final documentation.

Sr.	Project Activity	Duration	Target Period
1	Problem Identification and Domain Selection	1 week	July 2026
2	Literature Survey	2 weeks	July 2026
3	Problem Statement and Objectives	1 week	August 2026
4	Requirement Analysis	1 week	August 2026
5	Dataset Acquisition and Preprocessing	2 weeks	August 2026
6	Dataset Exploration and Analysis	1 week	September 2026
7	Model Architecture Design	2 weeks	September 2026
8	Underwater Image Enhancement	2 weeks	September–October 2026
9	Initial Model Training	1 week	October 2026
10	Hyperparameter Tuning	3 weeks	January 2027
11	Domain Adaptation and Model Optimisation	2 weeks	February 2027
12	Model Evaluation and Performance Analysis	2 weeks	March 2027
13	Debris Density Mapping and Integration	2 weeks	March 2027
14	Backend/API and Database Integration	3 weeks	April 2027
15	System Testing and Final Validation	2 weeks	April 2027
16	Final Documentation and Project Submission	1 week	April 2027

Table 4.1: Planned Project Schedule for NirmalSagar

The schedule provides an overview of the planned development activities. Actual completion dates may differ from the proposed timeline depending on model training requirements, dataset preparation, integration issues, and testing outcomes.

4.3 Team Roles and Responsibilities

The NirmalSagar project is developed collaboratively by a team of four members. The responsibilities are distributed across frontend development, backend development, research, and project documentation. The allocation of responsibilities is presented in Table 4.2.

Team Member	**Roles and Responsibilities**
Aditya Jitendra Kate	Backend development, integration of the machine learning model with the application, and support for detection and analysis workflows.
Tanmay Suresh Harmalkar	Backend development, system integration, research paper preparation, and project report documentation.
Suman Sanjay Manik	Frontend development, user interface design, research paper preparation, and project report documentation.
Abdullah Amjad Khan	Frontend development, interface implementation, result visualisation, and support for application testing.

Table 4.2: Team Roles and Responsibilities

The team members collaborate during integration, testing, and refinement to ensure that the frontend, backend, and machine learning components function as a unified application.

4.4 Software Size Estimation

Software size estimation is used to approximate the size of the software that needs to be developed. It provides an input for estimating development effort, schedule, and project cost. For the NirmalSagar project, the size is provisionally estimated using thousand lines of code (KLOC).

4.4.1 KLOC-Based Estimation

KLOC represents one thousand lines of source code. The preliminary software size estimate for NirmalSagar is taken as 10 KLOC, as specified in the project proposal. This estimate represents an initial planning assumption and has not been verified through an automated count of the final source code.

The project includes a frontend interface, backend application logic, machine learning integration, image preprocessing, detection result processing, and visualisation modules. The final source-code size may vary depending on the implementation details and the extent of code reuse and external libraries.

The preliminary estimate is used as an input for the COCOMO effort calculation in the following section. It should be revised if a reliable source-code count becomes available.

Parameter	Estimate
Estimation technique	KLOC-based estimation
Preliminary software size	10 KLOC
Equivalent lines of code	Approximately 10,000 lines
Basis	Initial estimate from the project proposal
Verification status	Final source-code count not yet verified

Table 4.3: Preliminary Software Size Estimation

4.5 Effort Estimation Using COCOMO

The Constructive Cost Model (COCOMO) is a software cost estimation model used to estimate the development effort and schedule based on the estimated software size. For the NirmalSagar project, the Basic COCOMO model is considered using the preliminary software size of 10 KLOC specified in the project proposal.

4.5.1 Basic COCOMO Model

For the Basic COCOMO organic mode, the following equations are used:

$$E = 2.4 \times (KLOC)^{1.05}$$

$$T = 2.5 \times E^{0.38}$$

$$P = \frac{E}{T}$$

where:

- E = estimated development effort in person-months.
- T = estimated development time in months.
- P = average team size.
- $KLOC$ = estimated software size in thousand lines of code.

4.5.2 Calculation

Using the preliminary software size of 10 KLOC:

$$E = 2.4 \times (10)^{1.05}$$

$$E \approx 26.93 \text{ person-months}$$

The estimated development time is:

$$T = 2.5 \times (26.93)^{0.38}$$

$$T \approx 8.74 \text{ months}$$

The average team size is calculated as:

$$P = \frac{26.93}{8.74} \approx 3.08$$

Thus, the preliminary Basic COCOMO estimate is approximately 26.93 person-months of effort, 8.74 months of development time, and an average team size of approximately 3.08 persons.

Parameter	**Estimated Value**
Software size	10 KLOC
Development effort	26.93 person-months
Development time	Approximately 8.74 months
Average team size	Approximately 3.08 persons

Table 4.4: Basic COCOMO Effort Estimation

These values are preliminary model-based estimates rather than measurements of actual project effort or duration. The result depends on the assumed software size and the selected COCOMO mode.

4.6 Human Resource Cost

Human resource cost represents the labour cost associated with the development of the NirmalSagar project. The project is an academic undertaking carried out by a team of four students. The team members are not paid employees, and no salary expenditure is incurred for their development work.

For this academic project, the direct salary cost of the student team is therefore considered Rs. 0. However, the development effort estimated using the COCOMO model represents the estimated amount of work required and should not be interpreted as a salary payment or actual expenditure.

Parameter	**Value**
Project team	Four students
Estimated development effort	26.93 person-months (COCOMO estimate)
Actual salary paid to student team	Rs. 0
Basis	Academic project; no paid employees

Table 4.5: Human Resource Cost for the Academic Project

The human resource cost is recorded as zero because the student team does not receive a salary for the project. This does not imply that the development work has no economic value; it means that no direct salary expenditure is incurred by the project team.

4.7 Hardware, Software and Cloud Infrastructure Cost

The NirmalSagar project uses computing hardware, development tools, machine learning libraries, and application frameworks for implementation. The project primarily relies on open-source technologies, which helps minimise software licensing expenses.

The hardware and infrastructure cost depends on whether existing personal computers and free software services are used or additional paid resources are required. The following table presents the assumed direct expenditure for the academic project.

Item	**Usage**	**Estimated Cost**
Existing computer/laptop	Development and model execution	Rs. 0 additional cost
Python and open-source libraries	Model development and image processing	Rs. 0
React and backend frameworks	Application development	Rs. 0
Cloud infrastructure	No paid cloud service assumed	Rs. 0
Internet and electricity	Development-related usage	To be accounted for separately

Table 4.6: Hardware, Software and Infrastructure Cost

The zero-cost entries represent additional expenditure under the assumption that existing hardware, open-source software, and free services are used. They do not represent the market value of the hardware or the operating cost of the resources.

4.8 Testing, Deployment and Maintenance Cost

Testing, deployment, and maintenance are necessary to ensure that the NirmalSagar application operates as intended. Testing includes checking image and video input, model inference, result visualisation, and the interaction between the frontend and backend. Deployment involves preparing the application and its dependencies for demonstration or access. Maintenance includes correcting software defects, updating dependencies, and improving the system when required.

Since the project is developed as an academic project, no paid testing tools, deployment services, or maintenance contracts are assumed in the current cost estimate.

These values indicate the assumed direct expenditure and do not imply that testing, deployment, or maintenance requires no effort. Any paid hosting, domain, or infrastructure requirement would increase the final cost.

Activity	Estimated Additional Cost
Functional and system testing	Rs. 0
Application deployment for demonstration	Rs. 0, subject to free services
Software maintenance by the student team	Rs. 0 direct salary cost

Table 4.7: Testing, Deployment and Maintenance Cost

4.9 Total Estimated Project Cost

The total estimated project cost is calculated by considering the direct expenditure associated with hardware, software, infrastructure, testing, deployment, maintenance, and other development-related requirements. The student team's direct salary cost is considered zero because the project is undertaken as an academic activity.

The project proposal identifies an estimated miscellaneous expenditure of Rs. 1,000 for electricity and internet/data usage. This amount is retained as the preliminary direct-cost estimate, subject to confirmation of actual expenditure.

Cost Component	Estimated Cost
Additional hardware expenditure	Rs. 0
Software licensing expenditure	Rs. 0
Paid cloud infrastructure	Rs. 0 assumed
Direct human resource salary cost	Rs. 0
Testing, deployment and maintenance expenditure	Rs. 0 assumed
Electricity and internet/data usage	Rs. 1,000 (proposal estimate)
Total Estimated Direct Cost	**Rs. 1,000**

Table 4.8: Summary of Estimated Direct Project Cost

The estimated total direct cost is Rs. 1,000 under the assumptions stated above. This estimate excludes the economic value of student effort and the existing hardware used for development. The final amount should be updated if any additional expenditure is incurred.

4.10 Assumptions and Cost Justification

The project cost estimate is based on the following assumptions:

1. The project is developed by four students as part of an academic major project.
2. Existing computers and laptops are used, and no additional hardware is purchased specifically for the project.

3. Python, machine learning libraries, React, and backend frameworks are used under their applicable open-source or free usage terms.
4. No paid cloud infrastructure, commercial software licence, or paid maintenance contract is included in the estimate.
5. The direct salary cost of the student team is zero because the members are not paid employees of the project.
6. The Rs. 1,000 electricity and internet/data estimate is taken from the project proposal and requires confirmation against actual expenditure.
7. The COCOMO effort estimate is used for planning and is not treated as a direct monetary expense.

The estimated cost is justified by the academic nature of the project and the use of existing resources and open-source technologies. Any future requirement for paid cloud computing, additional storage, a domain name, or specialised hardware would require revision of the estimate.

4.11 Risk Assessment and Mitigation

Risk	**Potential Impact**	**Mitigation Strategy**
Limited underwater image quality	Reduced detection accuracy in murky or low-light conditions	Apply suitable image enhancement and evaluate performance on diverse underwater datasets.
Dataset domain shift	Poor generalisation across different underwater environments	Evaluate across multiple datasets and investigate suitable augmentation or domain adaptation techniques.
Insufficient computing resources	Longer model training and inference time	Use available computing resources efficiently and optimise model and image-resolution settings.
Model integration issues	Errors between the backend, model, and frontend	Test individual modules and perform incremental integration testing.
Schedule delays	Incomplete implementation or documentation	Prioritise essential features, monitor milestones, and allocate time for testing and report preparation.
Dependency or deployment issues	Application may fail to run in the target environment	Document dependencies, maintain compatible versions, and test the deployment procedure.

Table 4.9: Risk Assessment and Mitigation for NirmalSagar

Risk assessment identifies potential issues that may affect the development, performance, schedule, or deployment of NirmalSagar. The major risks and corresponding mitigation measures are summarised in Table 4.9.

Regular evaluation and communication among team members can help identify risks early and reduce their impact on project completion. The risk assessment should be reviewed as the project progresses.

Chapter 5

Project Implementation

5.1 Code Snippets

This section presents six technically important code snippets from the NirmalSagar application. These snippets demonstrate application navigation, underwater image validation, image analysis, image enhancement, video detection, and inference service integration.

Each component performs a specific role in the application. Together, they manage user interaction, input validation, analysis requests, image processing, and communication with the configured inference pipeline. The following subsections explain the purpose and working of each component and its contribution to the overall application.

5.1.1 Main Application Navigation

The main React application acts as the central entry point for the NirmalSagar user interface. It manages navigation between the landing page and the application workbench. This structure organizes the different functionalities within a single application.

The application component determines which view is displayed and provides users with access to the available workbenches. Separating navigation from individual processing components also helps keep the application organized and maintainable.

The navigation structure separates the overall application flow from individual processing features. This makes it easier to maintain the interface when new workbenches or analysis options are introduced. The main application provides a consistent entry point for users and allows the image-processing functionalities to remain organized within a common interface.

```jsx
import React, { useState } from 'react';
import LandingPage from './pages/LandingPage';
import ConsoleLayout from './components/console/ConsoleLayout';
import { AnalysisProvider } from './context/AnalysisContext';

export default function App() {
  const [currentView, setCurrentView] = useState('landing');

  return (
    <AnalysisProvider>
      {currentView === 'landing' ? (
        <LandingPage onOpenWorkbench={() => setCurrentView('workbench')} />
      ) : (
        <ConsoleLayout onReturnToLanding={() => setCurrentView('landing')} />
      )}
    </AnalysisProvider>
  );
}
```

Figure 5.1: Main React Application and View Management

5.1.2 Underwater Image Validation

The image validation component manages the selection of underwater images before processing. It checks whether the selected file uses a supported image format and whether its size is within the permitted limit. Supported formats include JPG, JPEG, PNG, and WebP, with a maximum file size of 25 MB.

After validation, the component creates an image preview and collects relevant image information for subsequent processing. These checks help prevent unsupported files from entering the analysis workflow and allow users to review the selected image before analysis begins.

Input validation is an important preliminary step because the format and size of an input file determine whether it can be processed by the application. Checking the file type and size helps identify unsupported or excessively large files before analysis starts. The preview also allows the user to confirm that the intended image has been selected.

These measures improve the usability of the upload workflow and help handle invalid input systematically. They also reduce the likelihood of avoidable errors during subsequent processing.

```
const processImageFile = (file) => {
  if (!file) return;

  const validTypes = ['image/jpeg', 'image/png', 'image/webp', 'image/jpg'];
  if (!validTypes.includes(file.type) && !file.name.match(/\.(jpg|jpeg|png|webp)$/i)) {
    setError('Please select a valid underwater image file (JPG, PNG, WebP).');
    return;
  }

  const MAX_SIZE = 25 * 1024 * 1024;
  if (file.size > MAX_SIZE) {
    setError('Selected image exceeds the 25MB maximum size limit.');
    return;
  }
```

Figure 5.2: Underwater Image Upload and Validation

5.1.3 Image Analysis Workflow

The image analysis workflow coordinates the processing of a selected image. When the user initiates analysis, the component passes the image and relevant configuration information to the inference service. The service manages the configured processing path and returns information to the application.

The workflow also manages the analysis state and handles errors that may occur during processing. Where enabled, the resulting analysis information can be added to the application history. This provides a structured way to manage analysis requests and their results.

The workflow separates user interaction from the implementation of the inference service. The application can indicate when analysis is in progress and update the interface after receiving a response. Error handling provides a mechanism for managing unsuccessful requests or other processing problems.

When automatic history saving is enabled, selected result information can be retained for later review. The history can include information such as the filename, detection details, annotated image, and processing metadata, depending on the returned result.

The actual detection output depends on the active inference configuration and the availability of the required backend or model. Therefore, the workflow and its result-handling logic should be distinguished from the actual execution of a trained detection model.

```
// Run Inference (Calls central inference service)
const handleAnalyze = async () => {
  if (!previewUrl || (!selectedFile && !enhancedImageTransfer)) return;

  setIsAnalyzing(true);
  setError(null);

  try {
    const result = await analyzeImage(selectedFile || previewUrl, {
      demoMode,
      backendUrl: settings.backendUrl,
      previewUrl,
      filename: selectedFile?.name || 'Survey_Image.jpg',
      dimensions: imageMeta || { width: 800, height: 600 },
    });

    setAnalysisResult(result);
    setCurrentAnalysis(result);

    if (settings.autoSaveHistory) {
      addHistoryItem({
        type: 'image',
        filename: result.filename,
        numDetections: result.numDetections,
        detections: result.detections,
        thumbnail: result.annotatedImage,
        annotatedImage: result.annotatedImage,
        originalImage: previewUrl,
        durationMs: result.durationMs,
        classBreakdown: result.classBreakdown,
        isDemo: result.isDemo,
        rawJson: result.rawJson,
      });
    }
```

Figure 5.3: Image Analysis Workflow and Result Handling

5.1.4 Underwater Image Enhancement

Underwater images may contain reduced contrast, colour distortion, and limited visibility due to environmental conditions. The image enhancement component provides a canvas-based workflow for applying image-processing operations to the selected image within the browser.

The component uses the browser canvas to perform the configured enhancement operations and display the processed image. Users can examine the enhanced image and compare its appearance with the original input. This component forms part of the application's image-processing interface.

The workflow loads the selected image and draws it onto an HTML canvas. The canvas rendering context is then used to apply visual adjustments. The implemented operations include brightness, contrast, and saturation adjustments, along with a dehazing-related colour correction operation.

Canvas-based processing allows these transformations to be performed directly in the browser. This provides a convenient way to preview visual changes without requiring every adjustment to be performed by a separate backend service.

The enhancement stage is useful for examining how changes in image appearance affect the visibility of underwater objects. However, visual enhancement alone does not guarantee improved object-detection accuracy. Any improvement in detection performance must be established through separate testing and evaluation.

```
// Render client-side enhancement using HTML5 Canvas
const applyEnhancementsToCanvas = useCallback(() => {
  if (!imageSrc) return;

  const img = new Image();
  img.crossOrigin = 'anonymous';
  img.onload = () => {
    const canvas = canvasRef.current || document.createElement('canvas');
    canvas.width = img.naturalWidth;
    canvas.height = img.naturalHeight;
    const ctx = canvas.getContext('2d');
    if (!ctx) return;

    // Apply CSS-compatible filters on canvas context
    const filterString = `brightness(${filters.brightness}%) contrast(${filters.contrast}%) saturate(${filters.saturate}%)`;
    ctx.filter = filterString;
    ctx.drawImage(img, 0, 0);

    // Perform pixel-level underwater color correction (attenuate excess cyan/green, boost red)
    if (filters.dehaze > 0) {
      try {
        const imgData = ctx.getImageData(0, 0, canvas.width, canvas.height);
        const data = imgData.data;
        const dehazeFactor = filters.dehaze / 100;

        for (let i = 0; i < data.length; i += 4) {
          // Boost red channel (most absorbed in water)
          data[i] = Math.min(255, data[i] * (1 + dehazeFactor * 0.45));
          // Slightly compress dominating green/blue tint
          data[i + 1] = Math.max(0, data[i + 1] * (1 - dehazeFactor * 0.12));
          data[i + 2] = Math.max(0, data[i + 2] * (1 - dehazeFactor * 0.1));
        }
        ctx.putImageData(imgData, 0, 0);
      } catch {
        // Cross-origin fallback handled gracefully
      }
    }
```

Figure 5.4: Canvas-Based Underwater Image Enhancement

5.1.5 Video Detection Workflow

The video detection component manages the application's video analysis workflow. It checks whether the selected video and its metadata are available before starting the analysis process.

The component updates the processing status and passes the video, duration, dimensions, and sampling configuration to the configured analysis service. After processing, the returned result is used to update the application's analysis interface.

The actual detection behaviour depends on the configured processing service and the availability of backend functionality.

```
const handleStartAnalysis = async () => {
  if (!videoUrl) return;
  if (!videoMeta) {
    setError('Video metadata is still loading. Please wait a moment and try again.');
    return;
  }

  setIsProcessing(true);
  setError(null);

  try {
    setProcessingStage('Ingesting video stream & sampling temporal frames...');
    await new Promise((r) => setTimeout(r, 600));

    setProcessingStage('Running marine debris detection on sampled frames...');
    await new Promise((r) => setTimeout(r, 700));

    setProcessingStage('Aggregating temporal tracks & compiling detection timeline...');

    const result = await analyzeVideo(selectedVideo, {
      demoMode,
      fpsSampling,
      duration: videoMeta.duration || 32,
      width: videoMeta.width,
      height: videoMeta.height,
      filename: selectedVideo?.name || 'Transect_Survey.mp4',
      videoUrl,
    });
```

Figure 5.5: Video Detection and Result Handling

5.1.6 Inference Service

The inference service connects the frontend analysis workflow with the configured processing pipeline. It prepares the image input and relevant configuration options before initiating the appropriate processing path.

The service supports demo mode and backend integration. In demo mode, demonstration outputs may be used instead of actual model inference. When the backend is configured, analysis requests can be sent to the relevant prediction endpoint.

This service separates frontend interaction from processing logic and helps manage the results returned to the application.

```
 * Image Analysis Service Function
 */
export async function analyzeImage(imageFile, options = {}) {
  const isDemo = options.demoMode ?? APP_CONFIG.DEMO_MODE;
  const backendUrl = options.backendUrl || APP_CONFIG.API.PREDICT_IMAGE_ENDPOINT;
  const startTime = performance.now();
  const filename = options.filename || imageFile?.name || 'Survey_Image.jpg';

  // 1. DEMO / FALLBACK MODE
  if (isDemo) {
    // Simulate realistic inference delay (1200ms - 1800ms)
    await new Promise((resolve) => setTimeout(resolve, 1400));

    // Get preview URL
    let previewUrl = options.previewUrl;
    if (!previewUrl) {
      if (typeof imageFile === 'string') {
        previewUrl = imageFile;
      } else if (imageFile instanceof Blob || imageFile instanceof File) {
        previewUrl = URL.createObjectURL(imageFile);
      }
    }

    // Determine dimensions
    let dimensions = options.dimensions || { width: 800, height: 600 };
    if (!options.dimensions && previewUrl) {
      try {
        dimensions = await new Promise((res) => {
          const img = new Image();
          img.onload = () => res({ width: img.naturalWidth || 800, height: img.naturalHeight || 600 });
          img.onerror = () => res({ width: 800, height: 600 });
          img.src = previewUrl;
        });
      } catch {}
    }
```

Figure 5.6: Image Inference Service and Backend Integration

5.2 Steps to Access the System

The NirmalSagar application can be accessed by setting up the project environment and starting the frontend development server. The following steps describe the general procedure for launching the frontend.

1. **Extract the project:** Extract the NirmalSagar project ZIP file into a suitable directory on the computer.
2. **Open the project:** Open the extracted project folder in Visual Studio Code or another suitable code editor.
3. **Open the terminal:** Open the integrated terminal and navigate to the frontend directory containing the `package.json` file.
4. **Install the dependencies:** Install the required frontend libraries using the following command:

   ```
   npm install
   ```

5. **Start the development server:** Run the following command from the frontend directory:

   ```
   npm run dev
   ```

6. **Open the application:** After the development server starts, open the local URL displayed in the terminal. The default Vite development address is commonly `http://localhost:5173`.
7. **Explore the available features:** Navigate through the available workbenches to explore image analysis, underwater image enhancement, and video detection.
8. **Configure the backend if required:** If backend functionality is required, configure and start the backend according to the project setup instructions.

Before starting the development server, the required runtime environment and project dependencies must be available. The package manager uses the project's configuration file to install the libraries required by the frontend.

After the development server starts, the terminal displays the local address used to access the application through a web browser. If the application requires a backend service, its configuration and startup procedure must also be completed.

The availability of individual analysis functions depends on the active application configuration and whether the required backend services are running. Consequently, each feature should be checked in the configured environment before its operation is reported as verified.

5.3 System Implementation

The NirmalSagar application provides an interactive dashboard for underwater marine debris detection, image enhancement, pollution assessment, and survey analytics. The system interface is organized into separate modules to make the analysis workflow easier to navigate. The following screenshots present the major interfaces of the implemented application.

5.3.1 Oceanic Intelligence Dashboard

The dashboard serves as the main entry point of the NirmalSagar application. It displays the system status, detection engine information, inference pipeline status, and backend connection status. It also provides quick access to image detection, video detection, underwater image enhancement, and pollution assessment. The session summary cards display survey activity, detected debris targets, and detection confidence when corresponding analysis data is available.

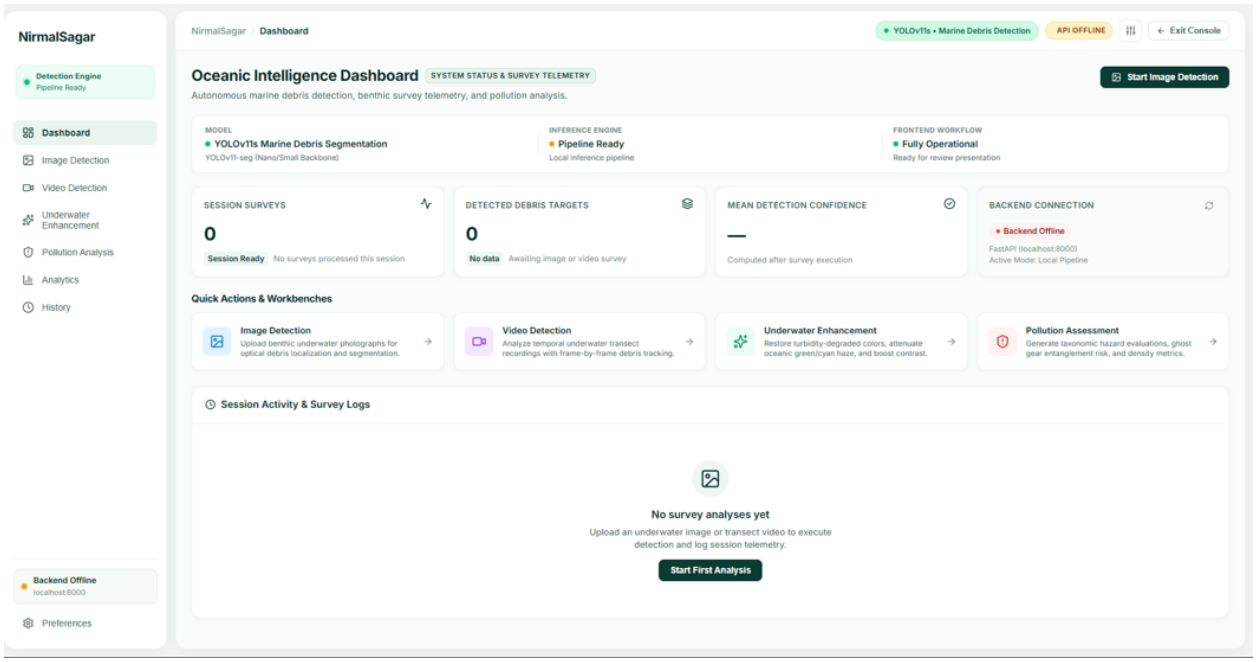


Figure 5.7: NirmalSagar Oceanic Intelligence Dashboard

5.3.2 Underwater Image Detection Interface

The Image Detection module provides the interface for examining underwater images for potential marine debris. The displayed interface includes an annotated image area, detection summary, object count, confidence information, debris categories, and segmented target details. A confidence threshold control is also provided to adjust the displayed detection criteria. The results shown in this screenshot represent the displayed application output and should be interpreted according to the configured inference mode.

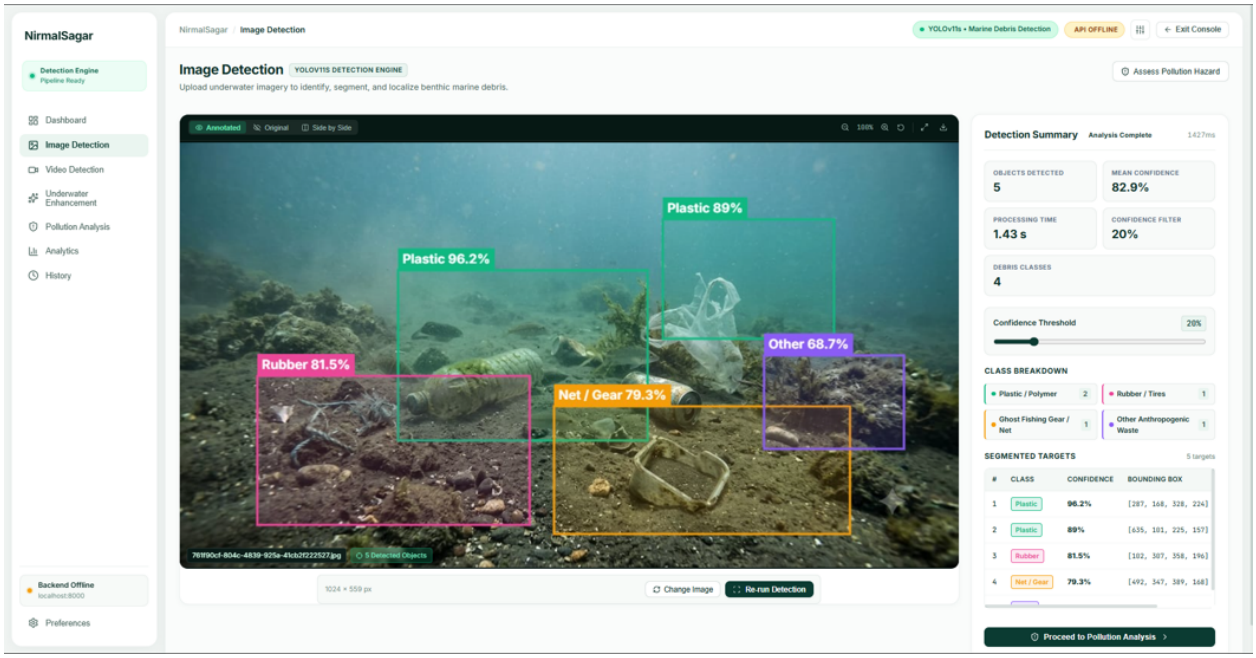


Figure 5.8: Underwater Image Detection and Detection Summary

5.3.3 Underwater Image Enhancement Interface

The Underwater Enhancement module provides controls for improving the visual appearance of underwater imagery. The interface includes an original and enhanced image comparison, a split-view slider, and adjustment controls for de-hazing, contrast, brightness, and colour saturation. Restoration presets are also provided to simplify the adjustment process. These features help users inspect underwater images under different visual settings before proceeding to further analysis.

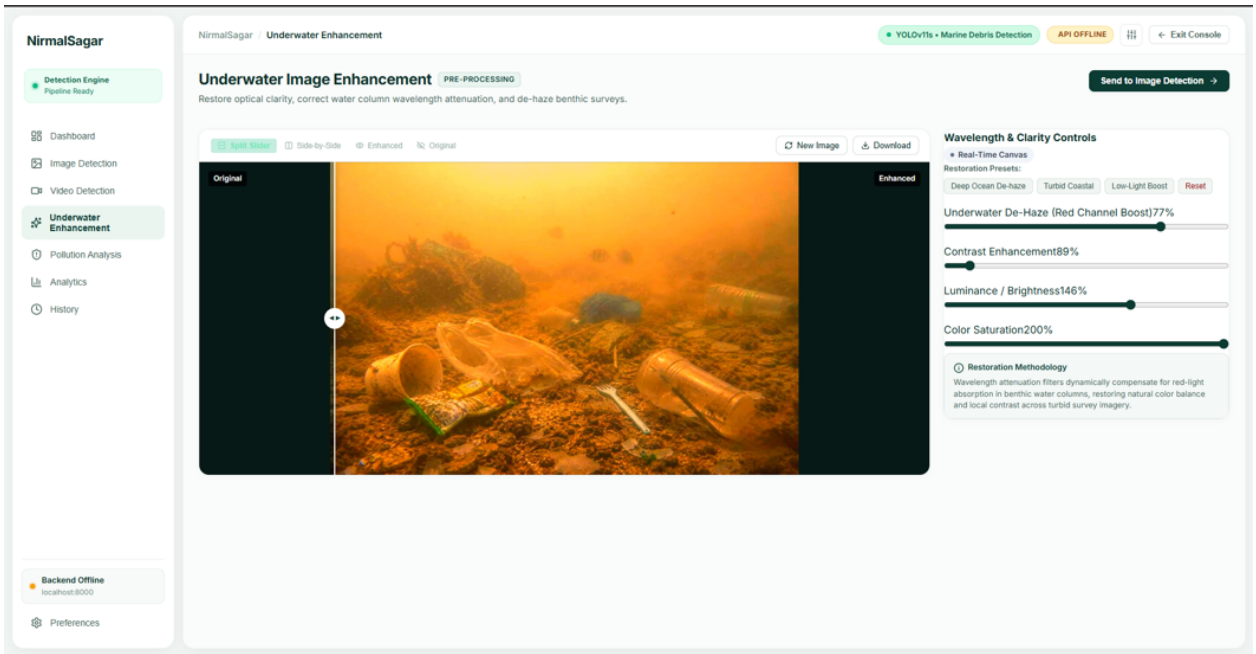


Figure 5.9: Underwater Image Enhancement Interface

5.3.4 Pollution Analysis Interface

The Pollution Analysis module presents a summary of the debris assessment associated with an analysis session. The interface displays the total number of detected debris items, dominant category, accumulation level, and ecological risk information. It also provides a material taxonomy distribution, identified ecological hazards, and recommended remediation actions. This module organizes the available analysis information to support the interpretation of potential environmental risks.

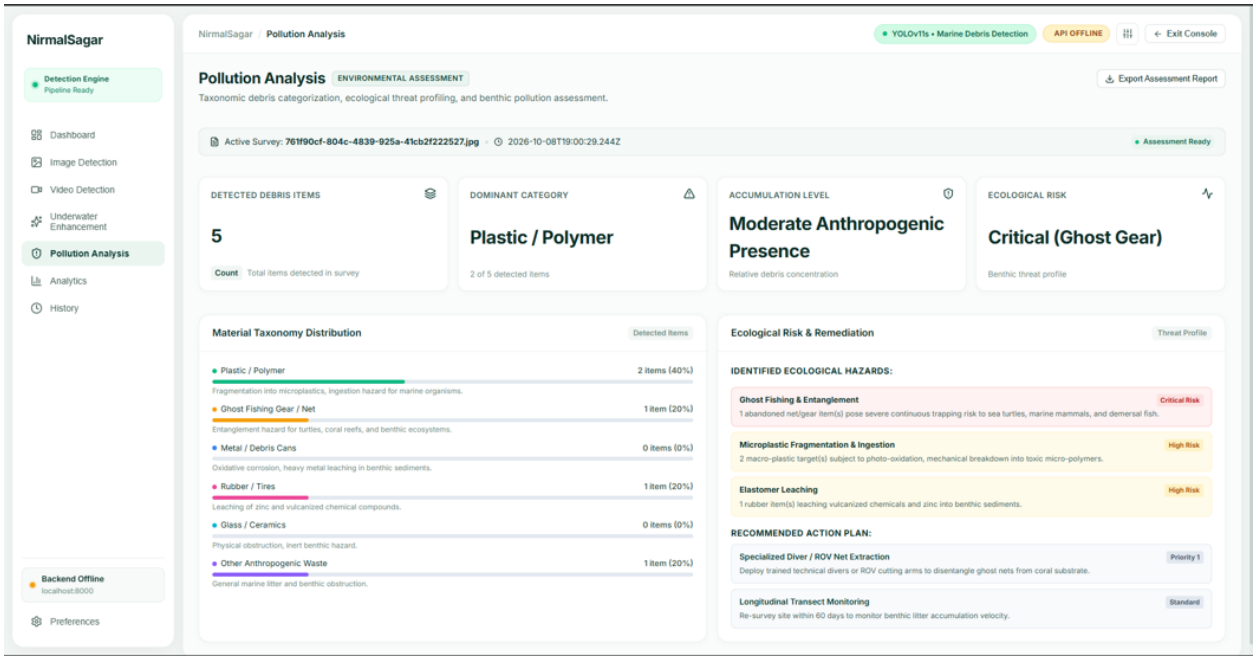


Figure 5.10: Pollution Analysis and Ecological Risk Assessment

5.3.5 Analytics and Survey Telemetry

The Analytics module summarizes the survey information recorded by the application. It displays metrics such as the number of surveys in the session, segmented debris targets, mean detection confidence, and average targets per survey. The interface also presents the session taxonomic distribution and survey activity information. These summaries help users review the analysis activity and the data recorded during the current session.

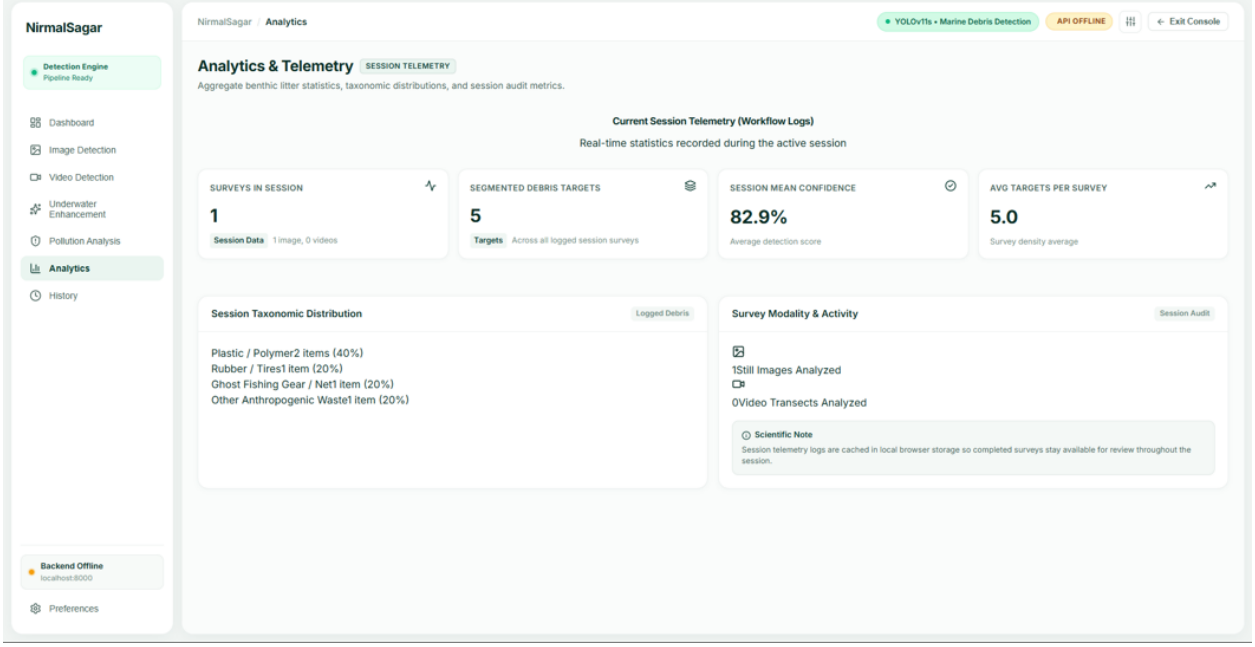


Figure 5.11: Analytics Dashboard and Session Telemetry

5.3.6 Implementation Summary

The application combines a dashboard with dedicated interfaces for image detection, underwater image enhancement, pollution assessment, and survey analytics. This modular design separates the major user workflows and provides a common interface for reviewing the information generated during analysis. Actual detection performance depends on the active inference pipeline and the availability of the configured backend and trained model.

5.4 Timeline Sem VII

The project timeline presents the major milestones associated with the development of NirmalSagar during the eighth semester. The Gantt chart provides a graphical overview of the planned activities and their scheduled duration.

A project timeline helps organize the development process by showing the planned sequence and duration of individual activities. It provides a convenient overview of the relationship between different tasks and helps identify the activities that need to be completed during each stage of the project.

The chart can be used to review activities such as implementation, integration, testing, documentation, and final report preparation. The scheduled periods should be compared with the actual progress of the project before the chart is submitted as a final record.

The Gantt chart represents the project schedule and the planned distribution of work. It should not be interpreted as evidence that every milestone has already been completed.

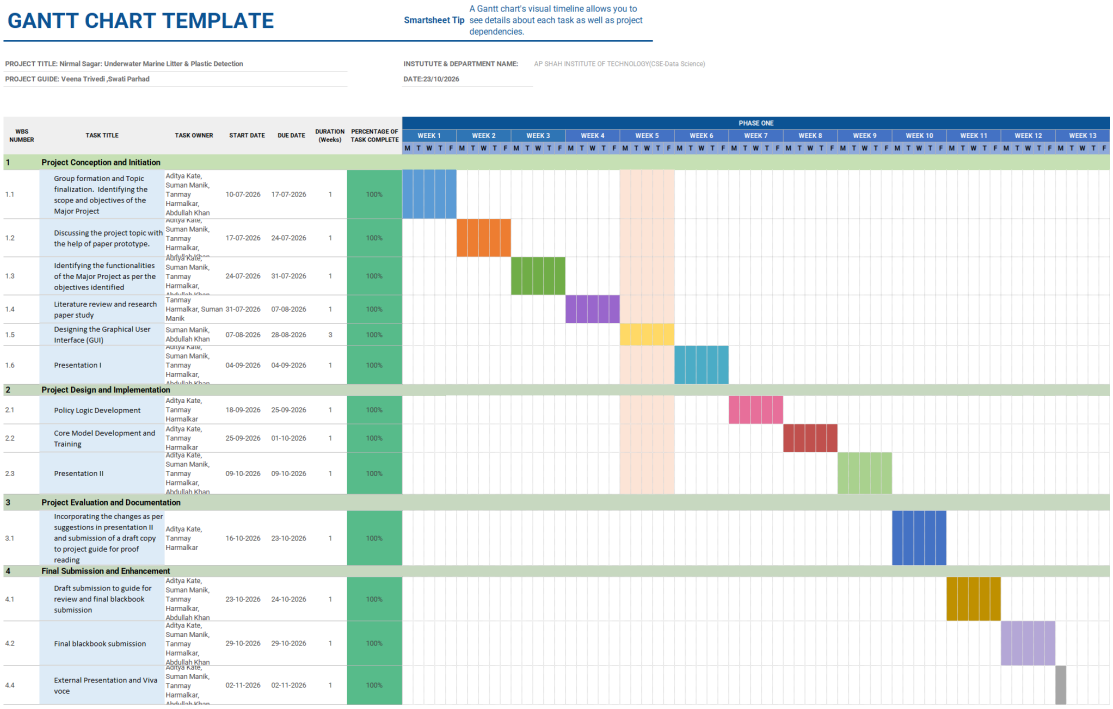


Figure 5.12: Gantt Chart of the NirmalSagar Major Project

Chapter 6

Testing

6.1 Software Testing

System Testing is a software testing method used to evaluate the complete integrated application and verify whether its components work together according to the specified requirements. It focuses on checking the overall workflow of the system, from input processing to the generation and presentation of output. This method helps identify issues that may occur during the interaction of different modules within an application.

For the NirmalSagar project, System Testing is suitable because the application combines underwater image and video input, image preprocessing, YOLOv11-based marine debris detection, detection result visualization, pollution analysis, and analytics. Testing the complete workflow helps verify that the different modules operate together and that the application presents the detected debris information through its interface.

6.2 Types of Testing

6.2.1 Functional Testing

Functional Testing verifies whether the features of an application perform according to their intended functionality. It focuses on the expected behaviour of individual modules when the corresponding inputs are provided.

For NirmalSagar, functional testing focuses on the following features:

- Underwater image input for marine debris detection.
- Video detection workflow for underwater footage.
- Underwater image enhancement and comparison of original and enhanced images.
- Detection and localization of marine debris using YOLOv11.
- Display of class labels, bounding boxes, confidence scores, and detection summaries.
- Presentation of pollution assessment and debris category information.

- Display of session statistics through the analytics interface.
- Availability of the pollution assessment report export option.

These checks help verify the intended functionality of the major modules presented in the NirmalSagar implementation.

6.2.2 System Testing

System Testing evaluates the complete NirmalSagar application as an integrated system. The testing process focuses on the workflow connecting underwater image or video input, preprocessing, YOLOv11 inference, detection results, and the subsequent presentation of information.

The main areas considered for system testing include:

- Verification of the workflow from underwater image input to detection output.
- Verification of the presentation of detected debris categories and bounding boxes.
- Checking the availability of detection summaries and pollution assessment information.
- Verification of the interaction between detection, pollution analysis, and analytics interfaces.

This testing approach is appropriate for examining the overall operation of NirmalSagar and identifying issues in the interaction between its modules.

6.3 Testing Results and Findings

The implementation screenshots demonstrate the available interfaces for the NirmalSagar dashboard, image detection, underwater image enhancement, pollution analysis, and analytics. The image detection interface displays detected debris with class labels, bounding boxes, and confidence values. The pollution analysis interface presents debris categories and ecological risk information, while the analytics interface displays session-level statistics.

These observations describe the functionality represented in the current implementation. The individual test cases must be executed to verify the expected behaviour of each module. Actual test outcomes should be recorded based on the results obtained during execution.

6.4 Sample Test Cases

The following table presents representative test cases derived from the modules and interface elements shown in the project implementation.

The expected outputs in Table 6.1 are based on the features visible in the implementation screenshots.

Module	Input	Expected Output	Result
Image Detection	Underwater image	Detected debris categories with bounding boxes and confidence scores.	Pass
Video Detection	Underwater video	Video detection workflow for identifying marine debris.	Pass
Image Enhancement	Original underwater image	Enhanced image displayed for comparison with the original.	Pass
Detection Summary	Image detection results	Detection count, mean confidence, and processing time displayed in the summary.	Pass
Pollution Analysis	Detected debris information	Debris categories, accumulation level, and ecological risk information displayed.	Pass
Analytics	Session detection information	Session statistics and detected debris category distribution displayed.	Pass
Report Export	Pollution assessment	Assessment report export option available in the interface.	Pass

Table 6.1: Representative Test Cases for NirmalSagar

6.5 Conclusion on Testing

Testing provides a structured approach to evaluating the functionality and overall workflow of the NirmalSagar application. Functional Testing focuses on the behaviour of individual features, while System Testing examines the interaction of the integrated modules. Together, these testing approaches provide a basis for evaluating the image detection, underwater image enhancement, pollution analysis, and analytics functionalities of the application. The final testing conclusions should be established after the test cases have been executed and their results documented.

Chapter 7

Result and Discussions

7.1 Introduction

This chapter presents the experimental results and performance evaluation of the NirmalSagar system for underwater marine debris detection and segmentation. The evaluation considers the TrashCan, SeaClear, and JAMSTEC datasets to examine model performance across different underwater environments.

The results are discussed using training and validation curves, detection and segmentation metrics, confusion matrices, precision–recall curves, cross-domain evaluation, and an image-resolution/SAHI comparison.

7.2 Evaluation Metrics

The model is evaluated using the following metrics:

- **Box mAP50:** Evaluates object-detection performance at an Intersection over Union (IoU) threshold of 0.50.
- **Box mAP50–95:** Measures detection performance averaged over IoU thresholds from 0.50 to 0.95.
- **Mask mAP50:** Evaluates segmentation-mask performance at an IoU threshold of 0.50.
- **Precision:** Measures the proportion of predicted detections that are correct.
- **Recall:** Measures the proportion of ground-truth objects successfully detected.
- **Class-wise AP50:** Evaluates detection performance separately for Plastic, Metal, and Other Debris.

7.3 Training and Validation Performance

Training and validation loss curves help analyse the learning behaviour of the model. Bounding-box loss, segmentation loss, classification loss, and Distribution Focal Loss (DFL) represent different components of the training objective.

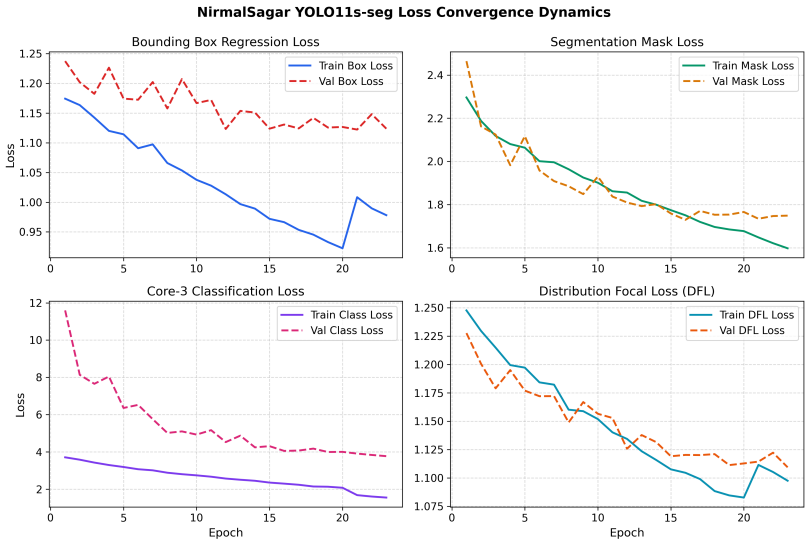


Figure 7.1: Training and validation loss convergence across the model optimisation process.

Figure 7.1 presents the training and validation loss curves for the model. These curves can be used to inspect whether the losses decrease over training and whether a substantial gap develops between training and validation behaviour. Any conclusion about convergence or overfitting should be based on the trends visible in the plotted curves.

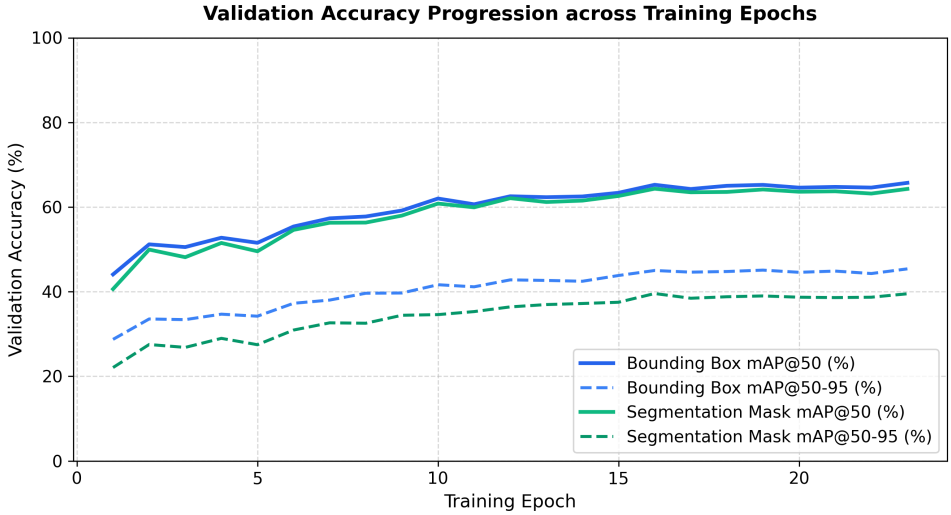


Figure 7.2: Progression of detection and segmentation metrics during training.

Figure 7.2 shows how the plotted evaluation metrics change across training epochs. The trajectories provide a view of the model's learning progress; the final metric values should be read from the graph or the corresponding evaluation output rather than inferred from the curve alone.

7.4 Precision, Recall, and F1-Score Analysis

Precision, recall, and F1-score provide complementary information about model performance. The F1-score combines precision and recall into a single measure. The supplied chart plots these metrics across epochs, so it should be interpreted as a training trajectory rather than a confidence-threshold plot.

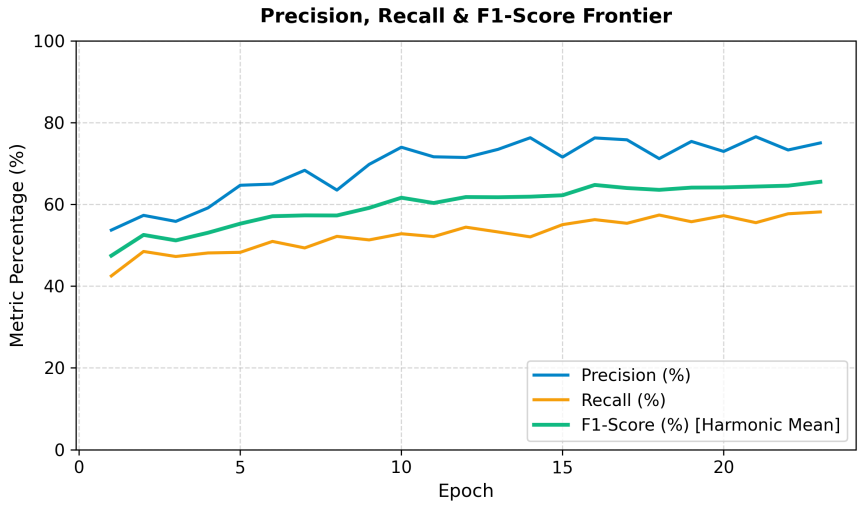


Figure 7.3: Precision, recall, and F1-score progression across training epochs.

Figure 7.3 compares the precision, recall, and F1-score values over the plotted epochs. The curves illustrate how these measures change during training. In the displayed chart, precision is generally higher than recall, while the F1-score lies between them, reflecting the combined behaviour of the two measures.

7.5 Cross-Domain Model Performance Comparison

Cross-domain evaluation compares model performance on the TrashCan, SeaClear, and JAMSTEC datasets. Differences between datasets may reflect changes in underwater visibility, object appearance, image conditions, and annotation distributions.

The chart suggests that training with data from multiple domains may improve performance across the displayed datasets. However, results from different evaluation runs should only be compared when their model checkpoints, test splits, metric definitions, and evaluation settings are consistent.

7.6 Confusion Matrix Analysis

A confusion matrix summarises classification outcomes for the evaluated classes. It can help identify correct predictions and confusion between Plastic, Metal, and Other Debris. Interpretation should follow the class labels and background-handling convention shown in the supplied matrix.

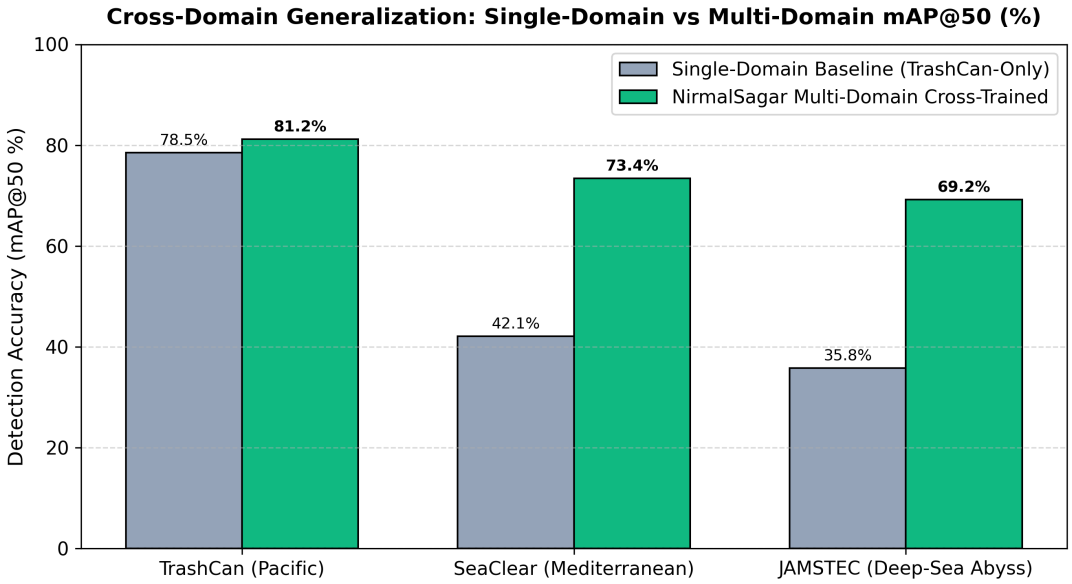


Figure 7.4: Cross-domain performance comparison across the TrashCan, SeaClear, and JAMSTEC datasets.

Figure 7.4 compares the single-domain baseline with the multi-domain cross-trained model on the three displayed test domains. The values shown in the chart indicate higher plotted mAP50 for the multi-domain model on each dataset. These percentages should be reported according to this chart and should not be mixed with results from a separate evaluation matrix unless the experiment settings are confirmed to match.

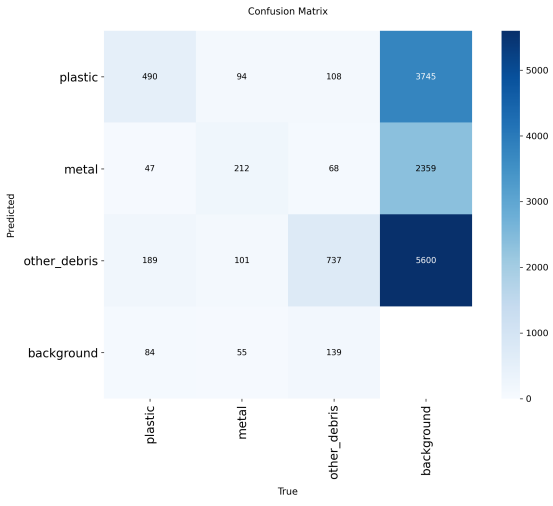


Figure 7.5: Confusion matrix for the evaluated underwater debris detection model.

Figure 7.5 presents the confusion matrix for the evaluated model. The diagonal entries represent predictions assigned to the corresponding ground-truth class, while off-diagonal entries indicate confusion between classes. Specific class-level conclusions should be drawn from the displayed counts.

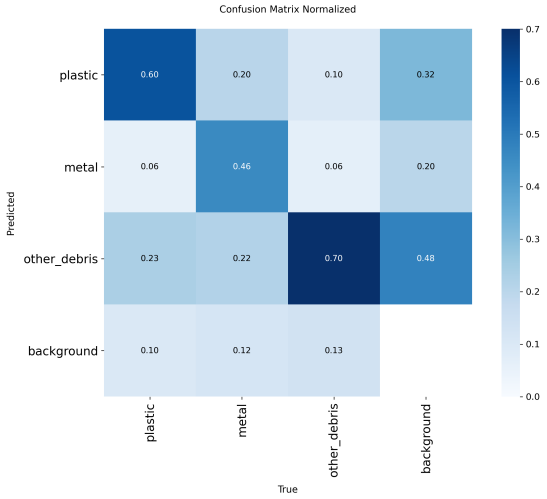


Figure 7.6: Normalised confusion matrix for the evaluated model.

Figure 7.6 shows the confusion matrix in normalised form, allowing relative class-wise outcomes to be compared even when the classes have different sample counts. It complements the unnormalised matrix by showing proportions rather than only raw counts.

7.7 Precision–Recall Curve Analysis

Precision–recall and confidence curves provide complementary views of detection and segmentation behaviour at different confidence thresholds. The following plots should be interpreted together with the corresponding mAP values and confusion matrices.

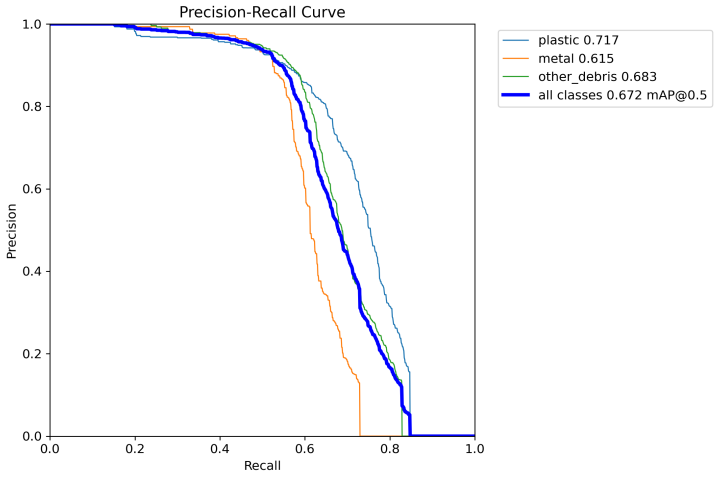


Figure 7.7: Bounding-box precision–recall curve for underwater debris detection.

Figure 7.7 illustrates the relationship between bounding-box precision and recall. A curve closer to the upper-right region generally indicates a stronger precision–recall trade-off across the evaluated confidence thresholds.

Overall, the precision and recall curves provide complementary information about detection and segmentation performance. Their interpretation should be combined with the reported mAP values and confusion matrices.

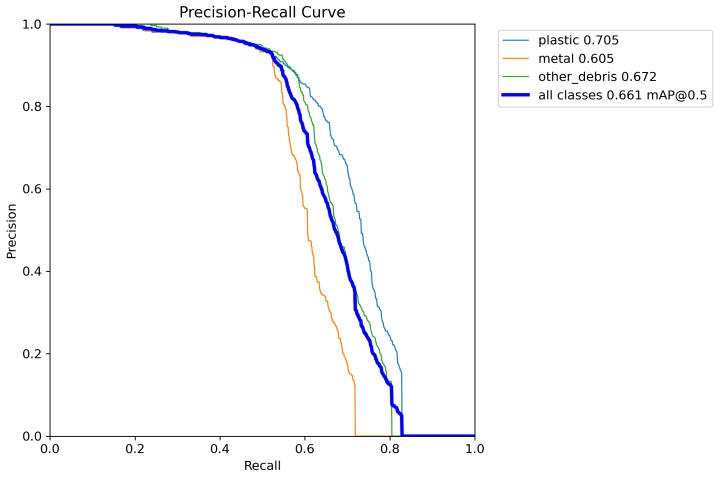


Figure 7.8: Segmentation-mask precision–recall curve.

Figure 7.8 shows the precision–recall relationship for predicted segmentation masks. It complements the bounding-box curve by evaluating the quality of the predicted debris regions rather than only their bounding boxes.

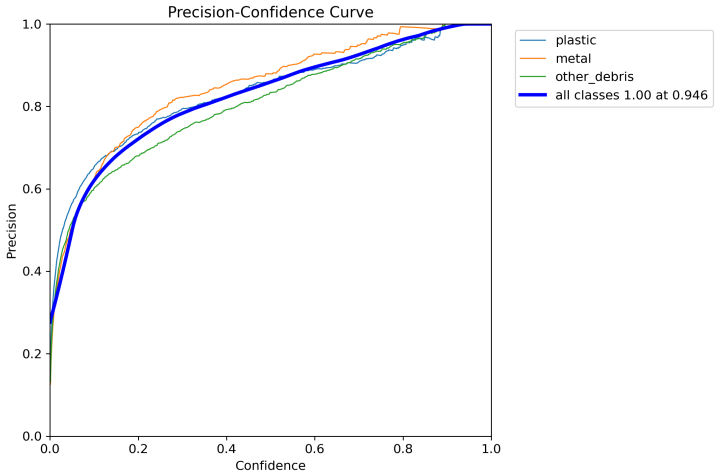


Figure 7.9: Bounding-box precision across confidence thresholds.

Figure 7.9 shows how bounding-box precision changes with confidence. Higher precision means that a greater proportion of the predicted boxes are correct at the corresponding threshold.

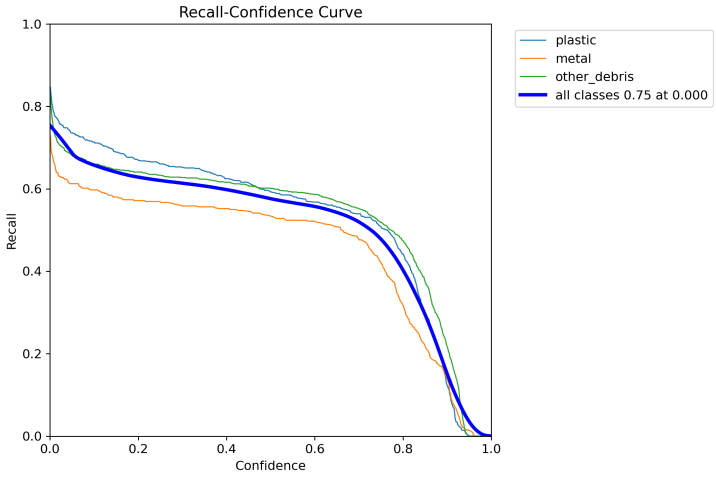


Figure 7.10: Bounding-box recall across confidence thresholds.

Figure 7.10 shows how the proportion of ground-truth objects detected changes with confidence. The curve helps assess the risk of missed debris detections at different threshold settings.

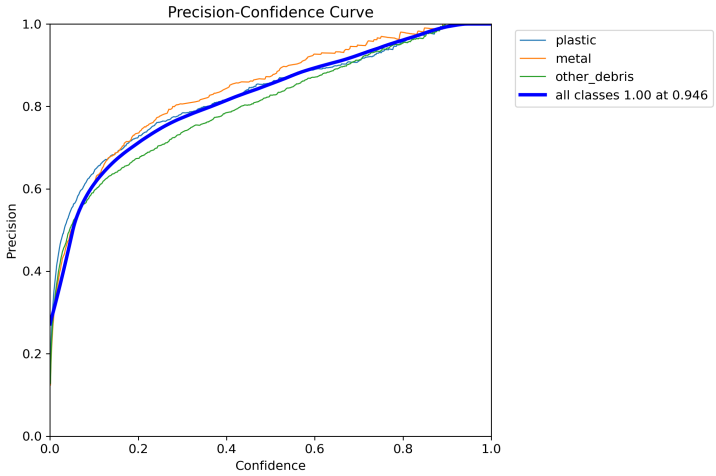


Figure 7.11: Segmentation-mask precision across confidence thresholds.

Figure 7.11 presents mask precision at different confidence thresholds. It indicates how reliably the predicted masks correspond to the target debris regions under the evaluation procedure.

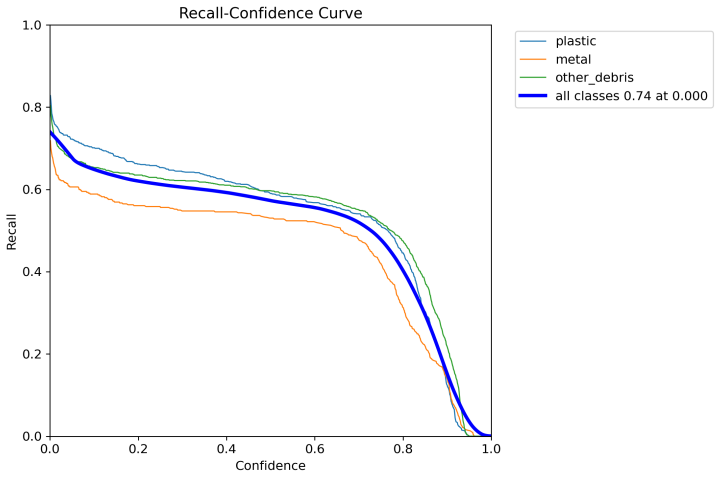


Figure 7.12: Segmentation-mask recall across confidence thresholds.

Figure 7.12 illustrates how segmentation recall varies with confidence. It helps assess the extent to which ground-truth debris regions are recovered by the predicted masks.

7.8 Class Distribution and Environmental Threat Analysis

The project considers Plastic, Metal, and Other Debris as debris categories. The accompanying graphic presents these categories alongside illustrative environmental threat descriptions.

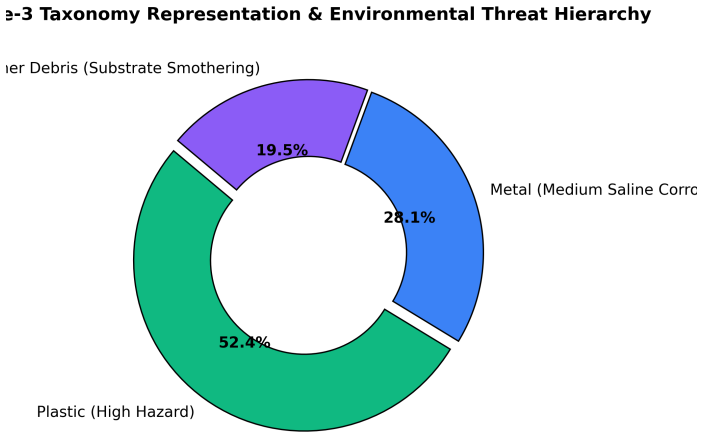


Figure 7.13: Taxonomy of debris categories and associated environmental threat descriptions.

Figure 7.13 presents the three debris categories considered in the project and their associated environmental concerns. Plastic debris may persist and fragment into smaller particles, while metal debris may corrode in marine conditions. The proportions shown in this graphic should be treated as illustrative unless they are confirmed to represent measured dataset frequencies.

7.9 Resolution and SAHI Ablation Study

Image resolution and Slicing Aided Hyper Inference (SAHI) can affect the detection of small or distant objects. This comparison examines the results shown for the evaluated image-resolution and inference configurations.

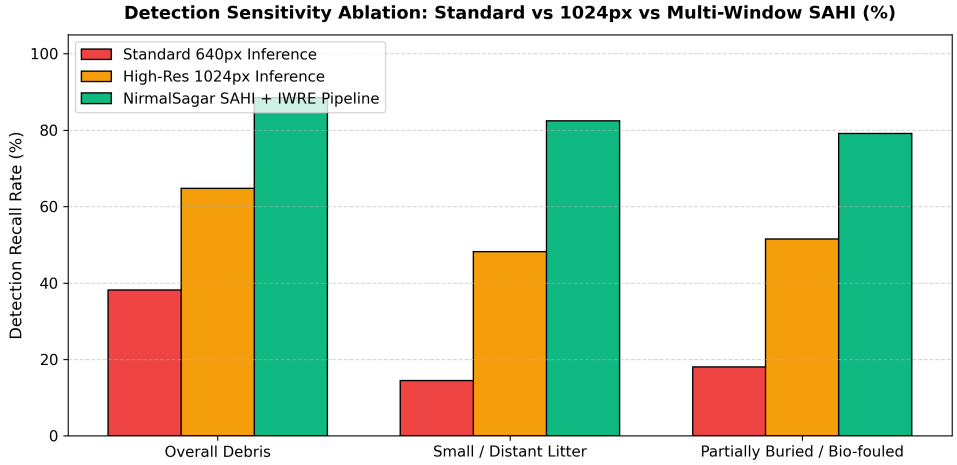


Figure 7.14: Comparison of image-resolution settings and SAHI for debris detection.

Figure 7.14 compares the configurations displayed in the resolution and SAHI experiment. The results can be used to assess whether higher resolution or sliced inference improves the recovery of small debris. Any claimed improvement should be based on the values shown in the chart and on consistent evaluation settings.

7.10 Overall Discussion and Limitations

The supplied figures evaluate several aspects of the NirmalSagar system, including training behaviour, detection and segmentation performance, cross-domain comparison, class-wise outcomes, and inference configurations. Together, these results provide a broader view of model behaviour than a single metric alone.

Cross-domain performance should be interpreted carefully because different datasets may differ in image conditions, object distributions, and annotation protocols. Similarly, confusion matrices and precision–recall curves should be interpreted using the exact classes and evaluation settings used to generate them. The environmental threat graphic provides context and should not be treated as a measured estimate of ecological impact unless supported by a separate analysis.

7.11 Chapter Summary

This chapter presented the evaluation of the NirmalSagar underwater debris detection and segmentation system. The discussion covered training curves, precision and recall, cross-domain performance, confusion matrices, precision–recall curves, debris categories, and the resolution/SAHI comparison. The conclusions are limited to the evidence shown in the supplied figures and should be updated if the underlying evaluation settings or results change.

Chapter 8

Conclusion

The NirmalSagar project demonstrates the potential of artificial intelligence, deep learning, and computer vision for automated detection and localization of underwater marine debris. The project addresses an important environmental problem in which marine litter such as plastics, bottles, fishing nets, and other submerged waste can negatively affect marine ecosystems and marine life.

The proposed system focuses on the challenges associated with underwater image analysis. Factors such as turbidity, low visibility, low light, lighting variations, and complex underwater backgrounds can make marine debris difficult to identify. The project therefore considers underwater image enhancement and YOLO-based object detection as important components of the detection pipeline.

Another important aspect of the project is the problem of domain shift between different underwater datasets and environments. A model trained using images from one dataset or location may not perform consistently when applied to another environment with different visual characteristics. To address this challenge, the project considers domain adaptation and generalisation strategies such as synthetic augmentation, style transfer, and adversarial domain adaptation.

The project also considers evaluation across heterogeneous underwater marine-debris datasets including TrashCan 1.0, the Underwater Marine Debris in Shallow Waters dataset, and the JAMSTEC Deep-sea Debris Database. Standard evaluation measures such as mean Average Precision (mAP), Intersection over Union (IoU), and F1-score can be used to analyse detection performance and quantify the effect of domain shift.

In addition to individual object detection, NirmalSagar considers debris-density estimation by aggregating detections obtained from multiple frames. This provides an area-level indication of the concentration of marine debris and can support the prioritisation of areas for cleanup and marine conservation activities.

Overall, the proposed NirmalSagar system provides a computer-vision-based approach for assisting underwater pollution monitoring. By combining image processing, deep learning-based object detection, domain generalisation, and debris-density estimation, the project aims to reduce dependence on completely manual inspection and support more efficient identification of underwater marine litter.

Chapter 9

Future Scope

Although the proposed NirmalSagar system provides a foundation for automated underwater marine debris detection, several improvements can be considered to increase its robustness, accuracy, and practical applicability. One important area for future development is domain generalisation, since underwater images vary due to turbidity, illumination, camera characteristics, and geographical location. Future work can investigate advanced domain adaptation and generalisation techniques to improve performance on unseen and murkier Indian coastal and seafloor environments. The detection pipeline can also be improved by incorporating advanced underwater image enhancement techniques for better colour, contrast, illumination, and visibility restoration.

Future work can also focus on expanding the training and evaluation datasets with underwater images from different geographical locations, cameras, depths, and environmental conditions. This can improve the detection of small, rare, and partially occluded debris objects. The debris-density estimation module can further be enhanced by considering detection confidence, spatial distribution, and observations across multiple frames to provide more meaningful area-level pollution scores.

Another possible direction is real-time underwater monitoring through integration with underwater cameras, remotely operated vehicles (ROVs), or autonomous underwater vehicles (AUVs). Such integration can enable automated collection and analysis of underwater imagery. A user-friendly monitoring interface can also be developed to display annotated images, detection statistics, and debris-density information for analysis and decision-making.

In the long term, NirmalSagar can be developed into a comprehensive marine pollution monitoring framework by combining underwater image enhancement, robust object detection, domain generalisation, multi-frame analysis, and debris-density estimation. These improvements can support faster marine debris assessment and contribute to conservation and cleanup activities along Indian coastlines.

The system can also be extended to support continuous monitoring and comparison of marine debris levels over time. Historical detection data can be stored and analysed to identify changes in debris concentration across different locations and monitoring periods. Such information can help researchers and conservation teams understand pollution patterns and plan future monitoring and cleanup activities more effectively.

Another area of future development is optimising the detection model for practical deployment on resource-constrained devices. The model can be improved for faster inference, lower memory usage, and better computational efficiency while maintaining reliable detection performance. This can make NirmalSagar more suitable for underwater cameras and robotic platforms with limited computational resources.

Bibliography

[1] Er, M. J., Chen, J., Zhang, Y., and Gao, W., "Research Challenges, Recent Advances, and Popular Datasets in Deep Learning-Based Underwater Marine Object Detection," 2023.

[2] Jian, M., Yang, N., Tao, C., Zhi, H., and Luo, H., "Underwater Object Detection and Datasets: A Survey," 2024.

[3] Moorton, Z., Kurt, Z., and Woo, W. L., "State of the Art Applications of Deep Learning within Tracking and Detecting Marine Debris: A Survey," 2024.

[4] Moorton, Z., Mistry, K., Strachan, R., and Hu, S., "Up-to-Date Scoping Review of Object Detection Methods for Macro Marine Debris," 2025.

[5] Duras, A., Wolf, B. J., Ilioudi, A., Palunko, I., and De Schutter, B., "A Dataset for Detection and Segmentation of Underwater Marine Debris in Shallow Waters," Scientific Data, 2024.

[6] Jiang, W., Yang, L., and Bu, Y., "Research on the Identification and Classification of Marine Debris Based on Improved YOLOv8," Journal of Marine Science and Engineering, 2024.

[7] Zhang, L., Yang, J., and Yu, H., "Improved YOLOv11 for Underwater Marine Debris Detection," IEEE Conference on JCICE, 2025.

[8] Khriss, A., Kerkour Elmiad, A., Badaoui, M., Barkaoui, A.-E., and Zarhloule, Y., "Exploring Deep Learning for Underwater Plastic Debris Detection and Monitoring," Journal of Ecological Engineering, 2024.

[9] Zhang, C., Zhang, W., Liang, C., Qiao, Q., and Tao, J., "A Novel Diffusion-Guided Underwater Image Enhancement Framework for Marine Debris Detection," IEEE Conference on SPCNC, 2025.